Contents

Introduction to React.js 4

History and evolution of React 4

Single Page Applications (SPA) concept 4

React ecosystem overview (React, ReactDOM, JSX, Babel, Webpack) 5

Hands-on: 5

Setting up development environment (Node.js, npm, create-react-app or Vite) 5

First React component 5

Components and Props 13

Functional vs Class components 13

Props and passing data between components 13

Component composition 14

Default props and prop types 15

Hands-on: 16

Part 1: Create Reusable Components & Pass Data 16

Part 2: Building the UI Component Library (The Main App) 17

State and Lifecycle 18

Introduction to useState hook 18

State management in functional components 18

Lifecycle in functional components with useEffect 19

Conditional rendering 20

Hands-on: 23

Interactive form with validation and Dynamic UI updates with state 23

Event Handling and Forms 25

Handling click, change, submit events 25

Controlled vs uncontrolled components 26

Form handling in React 26

Hands-on: 26

Build a multi-field form with state 26

Search bar component with live filtering 28

Lists, Keys, and Rendering 30
Rendering lists with map() 30
Using keys effectively 31
Conditional rendering patterns 32
Hands-on: 33
Build a to-do list app 33
Displaying dynamic data 34
Styling in React 36
Inline styles 36
CSS Modules 37
Styled-components 39
Tailwind CSS in React 39
Hands-on: 41
Apply different styling methods to the same component 41
React Router 43
Installing and configuring React Router 43
Route, Link, NavLink 45
Dynamic routes 46
Nested routes 46
Hands-on: 49
Build a Multi-Page Banking App 49
Advanced Hooks 52
useContext for global state 52
useReducer for complex state 56
Custom hooks 60
Hands-on: 64
Theme switcher with context 64
Build a custom data-fetching hook 66
Data Fetching 67
Fetch API and Axios 67

Async/await in React 73
Handling loading and error states 76
Hands-on: 77
Fetch data from a public API and Display with pagination 77
State Management with Redux or Zustand 80
Introduction to Redux concepts (store, actions, reducers) 80
Redux Toolkit basics 81
Zustand as a lightweight alternative 83
Hands-on: 85
Global state for cart system 85
Performance Optimization 88
React. memo, useCallback, useMemo 88
Code splitting and lazy loading 90
Avoiding unnecessary re-renders 92
Hands-on: 94
Optimize a slow React app 94
Testing in React 96
Jest basics 96
React Testing Library 98
Snapshot testing 101
Hands-on: 103
Write unit tests for components 103
Deploying React Apps 105
Building for production 105
Environment variables 105
Deploying to Netlify, Vercel, or GitHub Pages 106
Hands-on: 106
Deploy a working project to a hosting service (Netlify) 106
Deploy a working project to a hosting service (Vercel) 107

Introduction to React.js

React is a JavaScript **library** (not a full framework) for building user interfaces. Its "superpower" is the **Virtual DOM**, which allows it to update only the parts of a webpage that change, rather than reloading the whole thing.

Why use it?

1. **Component-Based:** Build once, reuse everywhere.
2. **Declarative:** You tell React *what* you want on the screen, and it handles the *how*.
3. **Speed:** The Virtual DOM makes updates lightning-fast.
4. **Community:** Huge ecosystem of pre-built tools.
5. **Job Market:** High demand for React developers globally.

History and evolution of React

Created by Jordan Walke at **Facebook**, React was first deployed on Facebook's News Feed in 2011. It went open-source in 2013 and revolutionized web dev by moving away from complex "templates" to "components."

Year	Milestone
2013	Initial Open Source release (JSConf US)
2015	React Native
2016	React 15
2019	React Hooks (v16.8)
2022	React 18 (Concurrent Rendering)
2023	Server Components ecosystem maturity
2024	React 19 (Actions, use, improved forms)
2025–2026	React Compiler + Framework-first architecture

Single Page Applications (SPA) concept

Unlike traditional websites where clicking a link loads a whole new .html file, an SPA loads **one** page and dynamically swaps out content. It feels like a desktop or mobile app.

Examples of SPAs:

1. **Gmail:** You click an email, the list disappears, and the message appears—no page refresh.
2. **Facebook:** Your feed updates while you’re typing in a chat box.
3. **Netflix:** Browsing movies feels like a continuous, fluid interface.
4. **Trello:** Dragging cards across lists happens instantly.
5. **Google Maps:** Panning and zooming happen without the browser "blinking."

React ecosystem overview (React, ReactDOM, JSX, Babel, Webpack)

React doesn't work alone. It's part of a team:

- **React:** The core logic (components, state).
- **ReactDOM:** The "glue" that renders React components into the browser.
- **JSX:** A syntax that looks like HTML but lives in JavaScript.
- **Babel:** A compiler that turns modern JS/JSX into code old browsers understand.
- **Vite/Webpack:** "Bundlers" that package all your files into one neat folder.

Hands-on:

Setting up development environment (Node.js, npm, create-react-app or Vite)

Vite has replaced create-react-app as the industry standard for speed.

1. Type
   ```
   npm create vite@latest my-react-app -- --template react
   ```
2. Type
   ```
   cd my-react-app
   ```
3. Run
   ```
   npm install
   ```
   (this downloads the "node_modules").
4. Run
   ```
   npm run dev
   ```
   Open the link provided (usually localhost:5173).

First React component

Components are just JavaScript functions that return "HTML-like" code.

1. The "Welcome" Heading (Basic Labels)

This shows how to create a simple text header.

- Create a new file in your src folder named Greeting.jsx.
- Paste this code:

```
function Greeting() {
  return <h1>Welcome to our first React class!</h1>;
}

export default Greeting;
```

2. The "Profile" Component (Nesting)

This shows how to put one component inside another.

- **Step 1:** Create Icon.jsx

```
function Icon() {
  return <span>👤</span>; // <span> is used for small inline items
}
export default Icon;
```

- Create Profile.jsx (notice how we "nest" the Icon):

```
import Icon from './Icon';

function Profile() {
  return (
    <div>
      <Icon />
      <p>This is a User Profile</p>
    </div>
  );
}
export default Profile;
```

3. Using Variables (Curly Braces)

This shows how to display dynamic text using { }.

- Create Status.jsx.
- Paste this code:

```
function Status() {
  const username = "Teacher Joe"; // This is a variable
  const date = "Monday";

  return (
    <div>
      <h3>Instructor: {username}</h3>
      <p>Today is: {date}</p>
    </div>
  );
}
export default Status;
```

4. Making a Functional Button

This shows how to create an interactive element.

- Create ActionButton.jsx.
- Paste this code:

```
function ActionButton() {
  return (
    // <button> creates a clickable button on the screen
    <button style={{ backgroundColor: 'blue', color: 'white' }}>
      Click to Enroll
    </button>
  );
}
export default ActionButton;
```

5. Returning Multiple Items (Fragments)

React requires you to return only **one** main item. If you have two items, you wrap them in "ghost tags" < > </ >.

- Create Info.jsx.
- Paste this code:

```
function Info() {
  return (
    <>
      <h2>Course: React 101</h2>
      <p>Duration: 4 Weeks</p>
      <p>Level: Beginner</p>
    </>
  );
}
export default Info;
```

To see any of these on your screen, you must "import" and "place" them inside your main **App.jsx** file.

1. Open App.jsx.
2. Replace everything in that file with this:

```
// 1. Import your custom components
import Greeting from './Greeting';
import Profile from './Profile';
import Status from './Status';
import ActionButton from './ActionButton';
import Info from './Info';

function App() {
  return (
    <div>
      {/* 2. Use them like custom tags */}
      <Greeting />
      <Status />
      <hr /> {/* This is a horizontal line */}
      <Profile />
      <Info />
      <br /> {/* This is a line break */}
      <ActionButton />
    </div>
  );
}

export default App;
```

JSX syntax basics

Before we write code, the React server needs to be running so it can instantly update the browser whenever you save a file.

1. Open your terminal or command prompt.
2. Navigate to the folder you created earlier: cd my-react-app.
3. Start the server by typing: npm run dev and press Enter.
4. It will give you a local link (usually http://localhost:5173). Leave this terminal window open and running in the background.

In React, the main file that gets shown on the screen is called App.jsx. We are going to use this file as our testing ground.

1. Open your project folder in your code editor (Cursor is perfect for this).
2. On the left-hand file menu, open the **src** folder.
3. Click on **App.jsx**.

To test examples, delete **everything** currently inside App.jsx. Your file should be completely blank.

```
function MathAndVariables() {
  const price = 100;
  const tax = 0.12;

  return (
    <div>
      <h1>Testing JSX!</h1>
      <p>The total price with tax is: {price + (price * tax)}</p>
    </div>
  );
}

// ADD THIS LINE AT THE BOTTOM
// This tells React to push this specific function to the screen.
export default MathAndVariables;
```

You can paste *any* function I gave you into this file. Just make sure the name in the export default line at the very bottom matches the name of the function you want to test.

Save and View

1. Save the file in your editor (Ctrl + S).
2. Look at your web browser (the localhost:5173 tab).
3. The page will have automatically refreshed, and you should instantly see:
   Testing JSX!
   The total price with tax is: 112

1. The Single Root Rule
React can only "export" one main container at a time. If you have multiple items, they must be wrapped inside one single parent tag (like a or a <section>).

Examples:

```
// Example A: Wrapped in a div
const Layout1 = () => <div><h1>Title</h1><p>Text</p></div>;

// Example B: Wrapped in a header tag
const Layout2 = () => <header><nav>Home</nav><h1>Logo</h1></header>;

// Example C: Wrapped in a main tag
const Layout3 = () => <main><section>Post 1</section><section>Post 2</section></main>;

// Example D: Wrapped in a footer tag
const Layout4 = () => <footer><p>Contact</p><p>Copyright</p></footer>;
```

2. Fragments (The Ghost Wrapper)
Use <> </> when you need to follow the "Single Root Rule" but don't want to add extra unnecessary "boxes" (divs) to your website's structure.

Examples:

```
// Example A: Two headings
const Frag1 = () => <><h1>Main</h1><h2>Sub</h2></>;

// Example B: List items
const Frag2 = () => <><li>Home</li><li>About</li></>;

// Example C: Text and a Button
const Frag3 = () => <><p>Are you sure?</p><button>Yes</button></>;

// Example D: Multiple spans
const Frag4 = () => <><span>First Name</span><span>Last Name</span></>;
```

3. JavaScript Expressions (The Curly Braces { })
Think of curly braces as a **portal**. Inside them, you are no longer writing "labels" (HTML); you are writing "math and logic" (JavaScript).

Examples:

```
// Example A: Math
const MathEx = () => <p>Total: {10 + 20}</p>;

// Example B: Variables
const name = "Alice";
```

```
const VarEx = () => <h1>Hello, {name}</h1>;

// Example C: Functions
const getStatus = () => "Active";
const FunEx = () => <p>System is: {getStatus()}</p>;

// Example D: Logic (Upper-casing text)
const LogicEx = () => <p>Shouting: {"hello".toUpperCase()}</p>;
```

4. The className Rule

In standard coding, class is used for styling. In React, because the word class is used to define JavaScript categories, we must use className instead.

Examples:

```
// Example A: Standard class
const Class1 = () => <div className="container">Content</div>;

// Example B: Multiple classes
const Class2 = () => <button className="btn btn-primary">Submit</button>;

// Example C: Dynamic class using variables
const activeStatus = "is-active";
const Class3 = () => <div className={activeStatus}>Menu</div>;

// Example D: Class on a heading
const Class4 = () => <h1 className="main-title">Welcome</h1>;
```

5. CamelCase for Attributes

If an attribute has two words, the first is lowercase and the second is Capitalized. This is the "React way" of writing labels.

Examples:

```
// Example A: Clicking
const Attr1 = () => <button onClick={() => alert('Hi')}>Click</button>;

// Example B: Changing an input
const Attr2 = () => <input onChange={() => console.log('Typed!')} />;

// Example C: Tab Index
const Attr3 = () => <div tabIndex="0">Focusable element</div>;

// Example D: Mouse Over
const Attr4 = () => <div onMouseOver={() => console.log('Hover!')}>Hover Me</div>;
```

6. Self-Closing Tags
Every tag must be finished. If a tag doesn't wrap around text (like an image or a line break), you must put a / before the final >.

Examples:

```
// Example A: Images
const Self1 = () => <img src="logo.png" alt="Logo" />;

// Example B: Line break
const Self2 = () => <div>Line 1 <br /> Line 2</div>;

// Example C: Inputs
const Self3 = () => <input type="password" />;

// Example D: Horizontal line
const Self4 = () => <div>Section 1 <hr /> Section 2</div>;
```

7. Inline Styles (The Double Curlies {{ }})
The first { opens the "JavaScript portal," and the second { creates a "Style Object." Note: Properties like font-size become fontSize (camelCase).

Examples:

```
// Example A: Color
const Style1 = () => <p style={{ color: 'blue' }}>Blue Text</p>;

// Example B: Font Size
const Style2 = () => <h1 style={{ fontSize: '50px' }}>Big Title</h1>;

// Example C: Multiple styles
const Style3 = () => <div style={{ backgroundColor: 'black', color: 'white' }}>Dark Mode</div>;

// Example D: Margin/Padding
const Style4 = () => <button style={{ marginTop: '20px', padding: '10px' }}>Spaced Button</button>;
```

8. Conditional Rendering (Ternary Operator)
This is an if/else statement inside your UI. It follows the pattern: Condition ? ResultIfTrue : ResultIfFalse.

Examples:

```
// Example A: Logged In status
const Cond1 = ({isLoggedIn}) => <div>{isLoggedIn ? "Welcome" : "Please Login"}</div>;

// Example B: Dark Mode toggle
const Cond2 = ({isDark}) => <div className={isDark ? "dark-theme" : "light-theme"}>Text</div>;
```

```
// Example C: Checking if a list is empty
const items = [];
const Cond3 = () => <div>{items.length > 0 ? "You have items" : "Empty Cart"}</div>;

// Example D: Admin access
const Cond4 = ({role}) => <p>{role === "admin" ? "Delete Access" : "View Only"}</p>;
```

9. Mapping (The List Loop)
Instead of writing <li> five times, we take a list of data and "map" it into tags. Every item in a map needs a unique key.

Examples:

```
// Example A: List of names
const Map1 = () => ["A", "B"].map(n => <li key={n}>{n}</li>);

// Example B: Navigation links
const Map2 = () => ["Home", "Shop"].map(link => <a key={link} href="#">{link}</a>);

// Example C: Buttons from an array
const Map3 = () => [1, 2, 3].map(num => <button key={num}>Page {num}</button>);

// Example D: Table rows
const Map4 = () => ["User1", "User2"].map(u => <tr key={u}><td>{u}</td></tr>);
```

10. Boolean, Null, and Undefined (The "Hidden" Values)
In JSX, the values true, false, null, and undefined are **not** printed to the screen. This is useful for hiding things.

Examples:

```
// Example A: False will show nothing
const Hide1 = () => <div>{false}</div>;

// Example B: Null is used to show nothing
const Hide2 = () => <div>{null}</div>;

// Example C: Using "&&" to show only if true
const Hide3 = ({showIcon}) => <div>{showIcon && <span>🌟</span>}</div>;

// Example D: Undefined shows nothing
const Hide4 = () => <div>{undefined}</div>;
```

Components and Props

Functional vs Class components

In the early days of React, developers had to use complex JavaScript "Classes" to build components. In 2019, React introduced **Hooks**, which allowed developers to use simple "Functions" instead.

Functional Components are the modern industry standard. They are shorter, easier to read, and faster to write.

Comparison:

```
// The Old Way (Class Component) - Harder to read
class Greeting extends React.Component {
  render() {
    return <h1>Hello Class!</h1>;
  }
}

// The Modern Way (Functional Component) - Simple and clean
function Greeting() {
  return <h1>Hello Class!</h1>;
}
```

Props and passing data between components

- "Props" is short for Properties. Think of them as **custom settings** for your components.
- Just like an HTML image tag has a src setting (<img src="photo.jpg" />), your custom React components can have custom settings like <UserCard name="John" />.
- Props always flow **downwards**: from a "Parent" component to a "Child" component.

Ways to Pass Props (Testing Sandbox)
Let's test this. Open your **App.jsx** file, delete everything, and paste this entire block of code.

```
// 1. THE CHILD COMPONENT: It "receives" the props inside the parentheses
function StudentInfo(props) {
  return (
    <div style={{ border: '1px solid black', padding: '10px', margin: '5px' }}>
      {/* We use props.propertyName to display the data */}
      <h3>Name: {props.name}</h3>
      <p>Age: {props.age}</p>
      <p>Status: {props.isPassing ? "Passing" : "Failing"}</p>
    </div>
  );
}
```

```
// 2. THE PARENT COMPONENT: It "sends" the props
function App() {
  return (
    <div>
      <h1>Student Roster</h1>

      {/* Example 1: Passing a String (use quotes) */}
      <StudentInfo name="John" age="20" isPassing={true} />

      {/* Example 2: Passing a Number (use curly braces for numbers) */}
      <StudentInfo name="Angel" age={22} isPassing={false} />

      {/* Example 3: Reusing the component with different data! */}
      <StudentInfo name="Louise" age={19} isPassing={true} />
    </div>
  );
}

// 3. Export the Parent
export default App;
```

- **How to test:** Save the file and check your browser. You will see three distinct student cards, all generated by reusing the exact same StudentInfo component.

Component composition

Composition is just putting components inside of other components, like Russian nesting dolls. Sometimes, you want a "Wrapper" component (like a custom window or a card) that can hold *anything* inside of it. We do this using a special prop called **children**.

How to Test in App.jsx:

```
// 1. The Wrapper Component
// We extract 'children' from the props. This represents whatever is put INSIDE the tags.
function WarningBox({ children }) {
  return (
    <div style={{ backgroundColor: 'yellow', border: '2px solid red', padding: '20px' }}>
      <strong>ATTENTION:</strong>
      <hr />
      {children} {/* The custom content goes here */}
    </div>
  );
}

// 2. The Main App
function App() {
```

```
  return (
    <div>
      {/* Notice how we open and close <WarningBox> and put HTML inside! */}
      <WarningBox>
        <p>The server will go down for maintenance at midnight.</p>
        <button>Acknowledge</button>
      </WarningBox>
    </div>
  );
}

export default App;
```

Default props and prop types

What happens if you forget to pass a prop? The space will just be blank. To fix this, you can set **Default Props** (backup values).

How to Test in App.jsx:

```
// We set a default value directly in the parentheses using the "=" sign
function Greeting({ studentName = "Guest", course = "React 101" }) {
  return <h2>Welcome, {studentName}! You are enrolled in {course}.</h2>;
}

function App() {
  return (
    <div>
      {/* Example A: We provide props, so it uses them */}
      <Greeting studentName="Sarah" course="Advanced JavaScript" />

      {/* Example B: We provide NOTHING, so it uses the defaults! */}
      <Greeting />
    </div>
  );
}

export default App;
```

Hands-on:

Miniature Banking Dashboard Component Library

Part 1: Create Reusable Components & Pass Data

Think of a Component Library as a box of custom-branded LEGO bricks. You build the brick once, and then you can use it hundreds of times with different colors or text.

Component 1: The StatusBadge
This component will be a small label that shows whether an account is "Active" or "Locked".

In your src folder, create a new file named **StatusBadge.jsx**.

```
// We expect one prop: 'isActive' (a true/false value)
function StatusBadge({ isActive }) {
  // We use a Ternary Operator (? :) to decide the color and text
  const badgeStyle = {
    padding: '5px 10px',
    borderRadius: '12px',
    color: 'white',
    fontSize: '12px',
    fontWeight: 'bold',
    backgroundColor: isActive ? 'green' : 'red' // Green if true, Red if false
  };

  return (
    // <span> is used for small, inline pieces of text
    <span style={badgeStyle}>
      {isActive ? "ACTIVE" : "LOCKED"}
    </span>
  );
}

export default StatusBadge;
```

Component 2: The AccountCard
This component will act as a container for a user's information. It will receive multiple props and even use our StatusBadge inside of it!

In your src folder, create a new file named **AccountCard.jsx**.

```
import StatusBadge from './StatusBadge'; // We bring in our first component!

// We expect three props: name, role, and accountStatus
function AccountCard({ name, role, accountStatus }) {
  return (
    // <div> acts as the "box" holding everything together
```

```
  <div style={{
    border: '1px solid #ccc',
    padding: '15px',
    margin: '10px 0',
    borderRadius: '8px',
    backgroundColor: '#f9f9f9'
  }}>
    <h3 style={{ margin: '0 0 5px 0' }}>Client: {name}</h3>
    <p style={{ margin: '0 0 10px 0', color: 'gray' }}>Role: {role}</p>

    {/* We pass the 'accountStatus' prop down into the StatusBadge */}
    <StatusBadge isActive={accountStatus} />
  </div>
 );
}

export default AccountCard;
```

Part 2: Building the UI Component Library (The Main App)

Now, let's assemble our dashboard. We will use the AccountCard multiple times, passing different data to it each time.

Open your **App.jsx** file, delete everything inside it.

```
import AccountCard from './AccountCard';

function App() {
 return (
  <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif', maxWidth: '400px' }}>
    <h2>Banking Administration</h2>
    <hr />

    {/* Reusing our Component Library with different data! */}
    <AccountCard
      name="Risa"
      role="Primary Account Holder"
      accountStatus={true}
    />

    <AccountCard
      name="Chiz"
      role="Joint Account Holder"
      accountStatus={true}
    />
```

```
    <AccountCard
      name="Pia"
      role="Corporate Beneficiary"
      accountStatus={false}
    />

    <AccountCard
      name="Bong"
      role="Trustee"
      accountStatus={true}
    />
  </div>
 );
}

export default App;
```

Save all three files (StatusBadge.jsx, AccountCard.jsx, and App.jsx). Look at your browser running on localhost:5173.

State and Lifecycle

State and **Effects** are what make your app *interactive*.

Introduction to useState hook

In regular JavaScript, if you change a variable, the screen doesn't automatically update. In React, we use a "Hook" called useState.

Think of useState as giving your component a **memory**. When a "State" variable changes, React automatically hits the refresh button on *just that specific component* to show the new data.

It always returns an array with two things: [theCurrentValue, theFunctionToChangeIt].

```
const [balance, setBalance] = useState(1000);
```

State management in functional components

- State should be treated as **read-only**. You never change the state directly (e.g., balance = 500 is WRONG).
- You must always use the setter function React gives you (e.g., setBalance(500) is RIGHT).

Quick and simple demo of the useState hook

```
import React, { useState } from 'react';

function Counter() {
  // 1. Initialize state: 'count' is the value, 'setCount' is the updater function.
  // We set the initial value to 0.
  const [count, setCount] = useState(0);

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h2>Current Count: {count}</h2>

      {/* 2. Update state: Calling setCount triggers a re-render with the new value. */}
      <button onClick={() => setCount(count + 1)}>
        Add 1
      </button>

      <button onClick={() => setCount(count - 1)} style={{ marginLeft: '10px' }}>
        Subtract 1
      </button>

      <button onClick={() => setCount(0)} style={{ marginLeft: '10px' }}>
        Reset
      </button>
    </div>
  );
}

export default Counter;
```

Lifecycle in functional components with useEffect

Sometimes a component needs to do something *after* it shows up on the screen—like fetching data from a database or starting a timer. We call these "Side Effects," and we handle them with the useEffect hook. The most important part of useEffect is the **Dependency Array** (the [] at the end). It tells React *when* to run the effect:

- [] (Empty array) = Run only ONE time when the component first appears.
- [balance] = Run every time the balance state changes.
- No array = Run after *every single render* (Dangerous! Can cause infinite loops).

Quick and simple demo of the useEffect hook in React.
This example updates the browser tab's title whenever a counter changes.

```
import React, { useState, useEffect } from 'react';

function TitleChanger() {
  const [count, setCount] = useState(0);

  // 1. The Effect: React runs this function after the component renders.
  useEffect(() => {
    // This updates the actual browser tab title
    document.title = `You clicked ${count} times`;

  // 2. The Dependency Array: React checks this array before running the effect again.
  }, [count]);

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h2>Look at your browser tab's title!</h2>
      <p>Current Count: {count}</p>

      <button onClick={() => setCount(count + 1)}>
        Click to Update Title
      </button>
    </div>
  );
}

export default TitleChanger;
```

Conditional rendering

We touched on this with the Ternary Operator (? :). Combined with useState, conditional rendering lets you build dynamic UIs where elements appear or disappear based on user actions.

Common Pattern (&&): If you only want to show something when a condition is true (and show nothing if it's false), use &&.

```
{isError && <p style={{color: 'red'}}>Transaction Failed!</p>}
```

Example 1: The "Early Return" (Handling Loading States)
This is the standard pattern for when you need to fetch data and show a loading screen until the data arrives.

```
import React, { useState, useEffect } from 'react';

function LoadingDemo() {
  const [isLoading, setIsLoading] = useState(true);
```

```
  useEffect(() => {
    // Simulate fetching data from an API taking 2 seconds
    const timer = setTimeout(() => {
      setIsLoading(false);
    }, 2000);

    // Cleanup the timer if the component unmounts
    return () => clearTimeout(timer);
  }, []);

  // 1. Conditional Rendering: Early return
  // If isLoading is true, React stops here and renders this UI.
  if (isLoading) {
    return <div style={{ padding: '20px' }}>⏳ Loading your data...</div>;
  }

  // If isLoading is false, React skips the 'if' block and renders this.
  return <div style={{ padding: '20px' }}>✅ Data successfully loaded!</div>;
}

export default LoadingDemo;
```

Example 2: The Logical AND && (Toggling Visibility)
Use this when you want to render an element *only* if a condition is true, and render absolutely nothing if it's false.

```
import React, { useState, useEffect } from 'react';

function ToggleDemo() {
  const [showSecret, setShowSecret] = useState(false);

  useEffect(() => {
    // Run a side effect every time the visibility changes
    if (showSecret) {
      document.title = "Secret Revealed!";
    } else {
      document.title = "React App";
    }
  }, [showSecret]);

  return (
    <div style={{ padding: '20px' }}>
      <button onClick={() => setShowSecret(!showSecret)}>
        {showSecret ? 'Hide' : 'Reveal'} Secret
      </button>
```

```
    {/* 2. Conditional Rendering: Logical AND */}
    {/* The <p> tag only renders if showSecret is strictly true */}
    {showSecret && (
      <p style={{ background: '#eee', padding: '10px', marginTop: '10px' }}>
        🤫 It's dangerous to go alone! Take this.
      </p>
    )}
  </div>
 );
}

export default ToggleDemo;
```

Example 3: The Ternary Operator ? : (Switching Between Two States)
Use this when you need to render one block of UI if a condition is true, and a completely *different* block of UI if it is false.

```
import React, { useState, useEffect } from 'react';

function StatusDemo() {
 const [isSuccess, setIsSuccess] = useState(false);

 useEffect(() => {
  // If the user triggers success, automatically clear it after 3 seconds
  if (isSuccess) {
   const timer = setTimeout(() => setIsSuccess(false), 3000);
   return () => clearTimeout(timer);
  }
 }, [isSuccess]);

 return (
  <div style={{ padding: '20px' }}>
   <button onClick={() => setIsSuccess(true)} disabled={isSuccess}>
    Simulate Submission
   </button>

   <div style={{ marginTop: '15px', fontWeight: 'bold' }}>
    {/* 3. Conditional Rendering: Ternary Operator */}
    {/* format: condition ? render_this_if_true : render_this_if_false */}
    {isSuccess
     ? <span style={{ color: 'green' }}>🎉 Form submitted successfully!</span>
     : <span style={{ color: 'gray' }}>Waiting for submission...</span>
    }
   </div>
  </div>
```

```
  );
}

export default StatusDemo;
```

Hands-on:

Interactive form with validation and Dynamic UI updates with state

Build a secure "Client Registration Form" for a banking dashboard using useState to track what the user types, useEffect to validate it dynamically, and conditional rendering to show error messages.

The ClientForm Component

In your src folder, create a new file named **ClientForm.jsx**.

```
// 1. We must import the hooks from React!
import { useState, useEffect } from 'react';

function ClientForm() {
  // 2. Set up our State (Component Memory)
  const [clientName, setClientName] = useState("");
  const [deposit, setDeposit] = useState(0);
  const [errorMessage, setErrorMessage] = useState("");
  const [isSuccess, setIsSuccess] = useState(false);

  // 3. The Effect: Watch the 'deposit' state and validate it instantly
  useEffect(() => {
    if (deposit < 0) {
      setErrorMessage("Deposit cannot be negative!");
    } else {
      setErrorMessage(""); // Clear the error if it's fixed
    }
  }, [deposit]); // <-- The dependency array. It watches 'deposit'

  // 4. The Submit Function
  const handleSubmit = (e) => {
    e.preventDefault(); // Stops the page from refreshing

    // Final validation check
    if (clientName === "") {
      setErrorMessage("Client name is required.");
      return;
    }
    if (deposit < 0) return;

    // If everything is good, show success!
```

```
  setIsSuccess(true);
};

return (
  <div style={{ border: '2px solid #00529B', padding: '20px', width: '300px', borderRadius: '8px' }}>
    <h3>Bangko Sentral - New Client</h3>

    {/* 5. Conditional Rendering: Show success or the form */}
    {isSuccess ? (
      <div style={{ color: 'green', fontWeight: 'bold' }}>
        Account created for {clientName} with ₱{deposit}!
      </div>
    ) : (
      <form onSubmit={handleSubmit}>

        <label style={{ display: 'block', marginBottom: '5px' }}>Client Name:</label>
        <input
          type="text"
          placeholder="e.g., Risa, Chiz, Imee"
          value={clientName} // Binds the input to our state
          onChange={(e) => setClientName(e.target.value)} // Updates state when typing
          style={{ width: '100%', marginBottom: '10px' }}
        />

        <label style={{ display: 'block', marginBottom: '5px' }}>Initial Deposit (₱):</label>
        <input
          type="number"
          value={deposit}
          onChange={(e) => setDeposit(Number(e.target.value))}
          style={{ width: '100%', marginBottom: '10px' }}
        />

        {/* Conditional Rendering: Only show the error paragraph if there is an error */}
        {errorMessage && (
          <p style={{ color: 'red', fontSize: '12px', marginTop: '0' }}>
            {errorMessage}
          </p>
        )}

        <button
          type="submit"
          style={{ backgroundColor: '#00529B', color: 'white', padding: '10px', width: '100%', border: 'none' }}
        >
          Register Client
        </button>
```

```
      </form>
    )}
   </div>
  );
}

export default ClientForm;
```

Test it in App.jsx

1. Open your **App.jsx** file.
2. Delete everything and paste this:

```
import ClientForm from './ClientForm';

function App() {
  return (
   <div style={{ padding: '40px', fontFamily: 'sans-serif' }}>
    <h2>Banking Administration Portal</h2>
    {/* Render our interactive form */}
    <ClientForm />
   </div>
  );
}

export default App;
```

View the Magic

1. Try typing a negative number into the deposit field. You will instantly see the warning pop up (thanks to useEffect and Conditional Rendering).
2. Delete the negative number, and the warning vanishes.
3. Try clicking "Register Client" without a name. It will stop you.
4. Finally, type "Bato" and a positive deposit, then submit. The entire UI swaps out to show a success message!

Event Handling and Forms

Handling click, change, submit events

"Events" are how our app listens to the user. Did they click a button? Did they type in a box? Did they hit Enter?

Rules of React Events:

1. **camelCase:** They are written as onClick or onChange (not onclick).
2. **Pass a Function:** You don't pass a string of code; you pass an actual JavaScript function.

3. **The e Object:** Events automatically give you an "Event Object" (usually named e). This object contains the data about what just happened (like e.target.value, which holds the text someone just typed).

Example:

```
// We pass an arrow function directly into the onClick event
<button onClick={(e) => alert("Button was clicked!")}>
  Click Me
</button>
```

Controlled vs uncontrolled components

It's all about who is in charge of the data inside a form input.

- **Controlled (The React Way):** React's useState is the "single source of truth." Every time the user types a single letter, React updates the state, and the state updates the input box. It is a closed loop.
- **Uncontrolled (The Old Way):** The HTML element manages its own data (like standard web design). You have to use a special tool called a ref to "reach in" and grab the data only when you hit submit.

Recommended to use **Controlled Components** 99% of the time. It allows for instant validation (like the negative deposit warning we built earlier).

Form handling in React

In traditional HTML, submitting a form reloads the entire webpage. In a React Single Page Application (SPA), we *never* want the page to reload.

The Standard Form Pattern:

1. Put an onSubmit event on the <form> tag itself (not the button).
2. Inside your submit function, the very first line must be e.preventDefault(). This stops the dreaded page reload.
3. Send your State data to your database (or display it).

Hands-on:

Build a multi-field form with state

Build a multi-field "Bank Employee Registration" form. Since there are multiple fields, we will create multiple useState hooks to track them.

Step 1: Create EmployeeForm.jsx
Paste this into a new file in your src folder:

```
import { useState } from 'react';
```

```
function EmployeeForm() {
 // We set up a Controlled State for each input field
 const [firstName, setFirstName] = useState("");
 const [role, setRole] = useState("Teller"); // Default dropdown value
 const [branch, setBranch] = useState("");

 const handleRegister = (e) => {
  e.preventDefault(); // Stop the page reload!

  // We now have all the data perfectly captured in our state variables
  alert(` Registered: ${firstName} \nRole: ${role} \nBranch: ${branch}` );

  // Optional: Clear the form after submitting
  setFirstName("");
  setBranch("");
 };

 return (
  <div style={{ padding: '20px', border: '1px solid gray', width: '300px' }}>
   <h3>Register Employee</h3>

   {/* The onSubmit goes on the form! */}
   <form onSubmit={handleRegister}>

    <label>First Name:</label>
    <input
     type="text"
     value={firstName}
     onChange={(e) => setFirstName(e.target.value)} // The Controlled Loop
     style={{ width: '100%', marginBottom: '10px' }}
    />

    <label>Role:</label>
    <select
     value={role}
     onChange={(e) => setRole(e.target.value)}
     style={{ width: '100%', marginBottom: '10px' }}
    >
     <option value="Teller">Teller</option>
     <option value="Manager">Manager</option>
     <option value="Loan Officer">Loan Officer</option>
    </select>

    <label>Branch Name:</label>
```

```
      <input
        type="text"
        value={branch}
        onChange={(e) => setBranch(e.target.value)}
        style={{ width: '100%', marginBottom: '10px' }}
      />

      <button type="submit" style={{ width: '100%' }}>Submit</button>
    </form>
  </div>
 );
}

export default EmployeeForm;
```

Search bar component with live filtering

Build a search bar that instantly filters a list of clients as the user types.

Step 1: Create ClientDirectory.jsx

```
import { useState } from 'react';

function ClientDirectory() {
 // 1. The State to hold whatever the user types into the search box
 const [searchQuery, setSearchQuery] = useState("");

 // 2. Our raw "Database" of clients (Using current PH Senators)
 const allClients = [
  "Loren",
  "Risa",
  "Lito",
  "Pia",
  "Imee",
  "Tito",
  "Win"
 ];

 // 3. The Magic: We filter the list based on the state!
 // This re-calculates instantly every time the user types a letter.
 const filteredClients = allClients.filter(client =>
  client.toLowerCase().includes(searchQuery.toLowerCase())
 );

 return (
  <div style={{ padding: '20px', maxWidth: '300px', fontFamily: 'sans-serif' }}>
```

```
    <h2>Client Directory</h2>

    {/* The Controlled Search Input */}
    <input
      type="text"
      placeholder="Search for a client..."
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      style={{ width: '100%', padding: '8px', marginBottom: '15px' }}
    />

    {/* Displaying the filtered results */}
    <ul style={{ listStyleType: 'none', padding: 0 }}>
      {filteredClients.length > 0 ? (
        filteredClients.map(client => (
          <li key={client} style={{ padding: '8px', borderBottom: '1px solid #eee' }}>
            👤 {client}
          </li>
        ))
      ) : (
        <li style={{ color: 'red' }}>No clients found matching "{searchQuery}"</li>
      )}
    </ul>
  </div>
 );
}

export default ClientDirectory;
```

Step 2: Test Both in App.jsx

To see them side-by-side, paste this into your App.jsx:

```
import EmployeeForm from './EmployeeForm';
import ClientDirectory from './ClientDirectory';

function App() {
 return (
  <div style={{ display: 'flex', gap: '40px', padding: '40px' }}>
    <EmployeeForm />
    <ClientDirectory />
  </div>
 );
}

export default App;
```

Go to your browser. Start typing "Lito" into the Search Box. Watch how "Loren", "Risa", and the others instantly disappear, leaving only "Lito" without you ever having to hit a submit button or reload the page.

Lists, Keys, and Rendering

Rendering lists with map()

In traditional JavaScript, you might use a for loop to go through a list. In JSX, we cannot use for loops inside our HTML labels. Instead, we use the JavaScript .map() function.

Think of .map() as a factory assembly line: it takes an array of data, loops through every single item, and transforms each one into a React Component or an HTML element.

Examples of map():

```
// Example 1: Mapping a simple array of strings (Senators)
const basicList = ["Risa", "Chiz", "Pia", "Bong"];
const Ex1 = () => <ul>{basicList.map(name => <li>{name}</li>)}</ul>;

// Example 2: Mapping an array of Objects (More realistic)
const objList = [{ name: "Loren", role: "Chair" }, { name: "Lito", role: "Member" }];
const Ex2 = () => (
  <div>
    {objList.map(person => <p>{person.name} is a {person.role}</p>)}
  </div>
);

// Example 3: Mapping data directly into Custom Components
// (Assuming we have a <ClientCard /> component)
const Ex3 = () => (
  <div style={{ display: 'flex' }}>
    {basicList.map(name => <ClientCard clientName={name} />)}
  </div>
);

// Example 4: Mapping into a Table format
const data = [{ id: 1, amount: 500 }, { id: 2, amount: 1200 }];
const Ex4 = () => (
  <table>
    <tbody>
      {data.map(row => <tr><td>{row.id}</td><td>₱{row.amount}</td></tr>)}
    </tbody>
  </table>
);
```

Using keys effectively

Whenever you use .map(), React will yell at you in the console if you don't provide a **key** prop.
Why? Because of the Virtual DOM. If you have a list of 100 items and you delete item #42, React needs to know *exactly* which one was deleted so it doesn't have to redraw all 100 items. The key is a unique ID tag for each item.

Keys must be unique, stable (they don't change), and predictable. Do not use random numbers for keys

Examples of Keys:

```
// Example 1: The BEST way - Using a unique ID from your database
const dbData = [{ id: "A12", name: "Imee" }, { id: "B99", name: "Tito" }];
const Key1 = () => dbData.map(user => <li key={user.id}>{user.name}</li>);

// Example 2: The PITFALL - Using the array index (Only do this if the list NEVER changes/reorders)
const staticList = ["Win", "Bong", "Risa"];
const Key2 = () => staticList.map((name, index) => <li key={index}>{name}</li>);

// Example 3: Keys go on the OUTERMOST element returned by the map
const Key3 = () => dbData.map(user => (
  // The key goes on the <div>, NOT the <h2> or <p> inside it!
  <div key={user.id} className="card">
    <h2>{user.name}</h2>
    <p>Status: Active</p>
  </div>
));

// Example 4: Keys with Fragments
// If you must use a Fragment in a loop, you cannot use the shorthand <> </>.
// You must use the full <React.Fragment> to attach a key.
import React from 'react';
const Key4 = () => dbData.map(user => (
  <React.Fragment key={user.id}>
    <dt>{user.name}</dt>
    <dd>Details go here</dd>
  </React.Fragment>
));
```

Conditional rendering patterns

We want our UI to react to different states (e.g., loading screens, empty states, error messages). Here are the four standard ways professional developers conditionally hide or show elements.

Examples of Conditional Patterns:

```
// Example 1: Early Return (Best for Loading or Errors)
// If it's loading, we return the loading screen and stop the rest of the function.
function EarlyReturn({ isLoading, data }) {
  if (isLoading) return <h2>Loading database...</h2>;
  return <div>{data}</div>;
}

// Example 2: The Ternary Operator `? :` (Best for IF / ELSE)
// "Condition ? Show If True : Show If False"
function Ternary({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <button>Log Out</button> : <button>Log In</button>}
    </div>
  );
}

// Example 3: Logical AND `&&` (Best for IF with NO ELSE)
// "If condition is true, show the thing. If false, do nothing."
function LogicalAnd({ notifications }) {
  return (
    <div>
      <h2>Inbox</h2>
      {notifications > 0 && <span className="badge">You have {notifications} new messages!</span>}
    </div>
  );
}

// Example 4: Conditional Styling
// Changing how something looks based on state, rather than adding/removing tags
function ConditionalStyle({ isOverdrawn }) {
  return (
    // If overdrawn, make background red. Otherwise, make it white.
    <div style={{ backgroundColor: isOverdrawn ? 'red' : 'white' }}>
      Account Balance
    </div>
  );
}
```

Hands-on:

Build a to-do list app

1. Create a new file in src named **TaskManager.jsx**.
2. Use this code:

```
import { useState } from 'react';

function TaskManager() {
 // 1. State for the input box
 const [newTask, setNewTask] = useState("");
 // 2. State for the list of tasks (starts as an empty array)
 const [tasks, setTasks] = useState([]);

 // 3. Function to add a task
 const handleAddTask = (e) => {
  e.preventDefault();
  if (newTask === "") return; // Stop if input is empty

  // Create a new task object with a unique ID (using current time)
  const taskObject = {
   id: Date.now(),
   text: newTask
  };

  // Add the new task to our existing array of tasks
  // We use the "Spread Operator" (...) to copy the old tasks, then add the new one
  setTasks([...tasks, taskObject]);
  setNewTask(""); // Clear the input box
 };

 // 4. Function to delete a task
 const handleDelete = (taskId) => {
  // Filter out the task that matches the ID we want to delete
  const updatedTasks = tasks.filter(task => task.id !== taskId);
  setTasks(updatedTasks);
 };

 return (
  <div style={{ padding: '20px', maxWidth: '350px', border: '2px solid black' }}>
   <h2>Office Tasks</h2>

   <form onSubmit={handleAddTask} style={{ display: 'flex', marginBottom: '20px' }}>
    <input
     type="text"
     value={newTask}
```

```
        onChange={(e) => setNewTask(e.target.value)}
        placeholder="New task..."
        style={{ flexGrow: 1, padding: '5px' }}
      />
      <button type="submit" style={{ padding: '5px 15px' }}>Add</button>
    </form>

    {/* CONDITIONAL RENDERING: Check if the list is empty */}
    {tasks.length === 0 ? (
      <p style={{ color: 'gray', fontStyle: 'italic' }}>No tasks pending! Good job.</p>
    ) : (
      <ul style={{ padding: 0, listStyle: 'none' }}>
        {/* MAPPING & KEYS: Loop through the tasks array */}
        {tasks.map((task) => (
          <li
            key={task.id}
            style={{ display: 'flex', justifyContent: 'space-between', padding: '10px', borderBottom: '1px solid
#ccc' }}
          >
            {task.text}
            <button onClick={() => handleDelete(task.id)} style={{ color: 'red' }}>X</button>
          </li>
        ))}
      </ul>
    )}
  </div>
 );
}

export default TaskManager;
```

Displaying dynamic data

1. Create a new file in src named **TransactionLedger.jsx**.
2. Use this code:

```
function TransactionLedger() {
 // Simulating data fetched from a database
 const transactions = [
  { id: "TX101", client: "Risa", amount: 5000, type: "Deposit", date: "Oct 1" },
  { id: "TX102", client: "Chiz", amount: -1500, type: "Withdrawal", date: "Oct 2" },
  { id: "TX103", client: "Pia", amount: 8000, type: "Deposit", date: "Oct 2" },
  { id: "TX104", client: "Bong", amount: -200, type: "Withdrawal", date: "Oct 3" },
 ];

 return (
```

```
  <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
    <h2>Recent Transactions</h2>

    <table style={{ width: '100%', borderCollapse: 'collapse', textAlign: 'left' }}>
      <thead>
        <tr style={{ backgroundColor: '#f0f0f0', borderBottom: '2px solid black' }}>
          <th style={{ padding: '10px' }}>ID</th>
          <th>Client</th>
          <th>Date</th>
          <th>Amount</th>
        </tr>
      </thead>
      <tbody>
        {/* Loop through our data to create table rows */}
        {transactions.map((tx) => (
          <tr key={tx.id} style={{ borderBottom: '1px solid #ddd' }}>
            <td style={{ padding: '10px' }}>{tx.id}</td>
            <td>{tx.client}</td>
            <td>{tx.date}</td>
            {/* Conditional Styling based on whether amount is positive or negative */}
            <td style={{
              color: tx.amount > 0 ? 'green' : 'red',
              fontWeight: 'bold'
            }}>
              ₱{Math.abs(tx.amount)}
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 );
}

export default TransactionLedger;
```

Test Both in App.jsx

Open your **App.jsx**, delete everything, and paste this:

```
import TaskManager from './TaskManager';
import TransactionLedger from './TransactionLedger';

function App() {
 return (
  <div style={{ display: 'flex', gap: '50px', padding: '40px', alignItems: 'flex-start' }}>
   {/* Our To-Do List */}
```

```
    <TaskManager />

    {/* Our Dynamic Data Table */}
    <TransactionLedger />
   </div>
  );
}

export default App;
```

Styling in React

Inline styles

We touched on this earlier. You use the "double curlies" {{ }} to pass a JavaScript object directly to the style tag.

- **Pros:** Extremely fast for prototyping; great for styles that change based on state (like a progress bar filling up).
- **Cons:** Makes your JSX file very long and messy; cannot use advanced CSS features like "hover" effects or media queries.

Examples of Inline Styling:

```
// Example 1: Basic static styling
const Box1 = () => <div style={{ backgroundColor: 'blue', padding: '20px' }}>Box</div>;

// Example 2: Dynamic styling based on variables
const radius = '8px';
const Box2 = () => <div style={{ borderRadius: radius, border: '1px solid black' }}>Box</div>;

// Example 3: Conditional inline styling
const isDanger = true;
const Box3 = () => <div style={{ color: isDanger ? 'red' : 'green' }}>Alert</div>;

// Example 4: Extracting the style object to keep JSX clean
const myStyles = {
  fontSize: '24px',
  fontWeight: 'bold',
  textAlign: 'center'
};
const Box4 = () => <h1 style={myStyles}>Bank Dashboard</h1>;
```

CSS Modules

In normal web design, if you create a class called .button in one file, it might accidentally overwrite a .button in another file. CSS Modules fix this by automatically generating unique, randomized class names for every component.

Examples of CSS Modules Workflow:

1. The Basic Setup (React/Vite)

To use CSS Modules, you simply name your file with the .module.css extension. This tells the bundler to treat it as a module rather than a global stylesheet.

The CSS File (Button.module.css)

```
.error {
  background-color: red;
  color: white;
  padding: 10px;
  border-radius: 5px;
}
```

The Component (Button.jsx)

```
import styles from './Button.module.css';

export function ErrorButton() {
  // Styles are accessed as properties of the imported object
  return <button className={styles.error}>Delete Everything</button>;
}
```

2. Handling Multiple Classes

When you need to apply more than one class (e.g., a base style and a specific modifier), you can use template literals.

The CSS File (Card.module.css)

```
.base {
  padding: 20px;
  border: 1px solid #ccc;
}

.highlight {
  border-color: gold;
  box-shadow: 0 0 10px gold;
}
```

The Component (Card.jsx)

```
import styles from './Card.module.css';

export function Card({ isFeatured }) {
  return (
    <div className={` ${styles.base} ${isFeatured ? styles.highlight : ''}` }>
      Simple Card Content
    </div>
  );
}
```

3. Using Global Classes Within a Module

Sometimes you want a specific class to remain "global" (unhashed) even though it is inside a CSS Module file. You use the :global selector for this.

The CSS File (Navbar.module.css)

```
/* This gets a unique name like _nav_1a2b3 */
.nav {
  display: flex;
  background: #333;
}

/* This stays exactly "active" in the final HTML */
:global(.active) {
  color: cyan;
  font-weight: bold;
}
```

The Component (Navbar.jsx)

```
import styles from './Navbar.module.css';

export function Navbar() {
  return (
    <nav className={styles.nav}>
      <a href="/" className="active">Home</a> {/* Note: No 'styles.' prefix needed for global */}
      <a href="/about">About</a>
    </nav>
  );
}
```

Styled-components

This requires installing a third-party library (npm install styled-components). It allows you to write actual CSS directly inside your JavaScript file to create custom, pre-styled components. It is very popular in enterprise applications.

- **Pros:** Keeps components and their styles in one single file without using inline objects.
- **Cons:** Requires learning a slightly different syntax (tagged template literals).

Examples of Styled-Components:

```
import styled from 'styled-components';

// Example 1: Creating a basic styled button
const StyledBtn = styled.button`
  background-color: navy;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
  &:hover { background-color: blue; } /* You can use hover states here! */
`;

// Example 2: Using the custom component
const App1 = () => <StyledBtn>Click Me</StyledBtn>;

// Example 3: Dynamic styling based on props
const AlertText = styled.p`
  color: ${props => props.isError ? 'red' : 'black'};
  font-weight: bold;
`;

// Example 4: Using the dynamic component
const App2 = () => <AlertText isError={true}>Warning: Insufficient Funds for Imee</AlertText>;
```

Tailwind CSS in React

Tailwind gives you thousands of pre-made, tiny class names (utilities). You stack them together to build complex designs without ever leaving your HTML/JSX.

- **Pros:** Unbeatable speed; incredible for responsive design (mobile vs desktop); no need to invent class names.
- **Cons:** Your className tags can get very long.

Install and Configure Tailwind CSS

```
# 1. install
npm install tailwindcss @tailwindcss/vite

# 2. config vite.config.js
...
import tailwindcss from '@tailwindcss/vite'
...
...
plugins: [
...
tailwindcss(),
]
...

# 3. config index.css, add this on top
@import "tailwindcss";

# test
...
<h1 className="text-3xl font-bold underline text-blue-600>
```

Check Tailwind CSS Classes here:
https://flowbite.com/tools/tailwind-cheat-sheet/
https://tailwind.build/classes

Examples of Tailwind Power:

```
// Example 1: Spacing and Colors
const Tail1 = () => <div className="p-4 bg-gray-100 text-blue-800">Card</div>;

// Example 2: Hover and Focus states directly in the class
const Tail2 = () => <button className="bg-blue-500 hover:bg-blue-700 text-white p-
2">Submit</button>;

// Example 3: Responsive Design (sm: means "on small screens and up")
const Tail3 = () => <div className="text-sm sm:text-xl lg:text-3xl">Responsive Text</div>;

// Example 4: Layouts (Flexbox)
const Tail4 = () => <div className="flex justify-between items-center"> <span>Left</span>
<span>Right</span> </div>;
```

Hands-on:

Apply different styling methods to the same component

For this lab, we will create four different files. Each file will render the exact same **"Client Alert Card"** for Senator Chiz, but each will use a completely different styling method.

Method 1: Inline Styles

1. Create AlertInline.jsx.
2. Paste this code:

```
function AlertInline() {
  const cardStyle = {
    backgroundColor: '#fff3cd',
    border: '1px solid #ffeeba',
    padding: '15px',
    borderRadius: '8px',
    color: '#856404',
    fontFamily: 'sans-serif',
    marginBottom: '10px'
  };

  return (
    <div style={cardStyle}>
      <strong>Notice:</strong> Please update KYC documents for client: Chiz.
    </div>
  );
}
export default AlertInline;
```

Method 2: CSS Modules

1. Create a new file called **Alert.module.css** (the .module.css part is required!).
2. Paste this standard CSS into it:

```
.cardBox {
  background-color: #d4edda;
  border: 1px solid #c3e6cb;
  padding: 15px;
  border-radius: 8px;
  color: #155724;
  font-family: sans-serif;
  margin-bottom: 10px;
}
```

3. Create AlertModule.jsx and paste this code:

```
import styles from './Alert.module.css'; // Import the CSS file as 'styles'

function AlertModule() {
  return (
    <div className={styles.cardBox}>
      <strong>Success:</strong> Deposit cleared for client: Loren.
    </div>
  );
}
export default AlertModule;
```

Method 3: Styled-Components

(Note: must run npm install styled-components in terminal first).

1. Create AlertStyled.jsx.
2. Paste this code:

```
import styled from 'styled-components';

// Define the styled element outside of the main function
const StyledCard = styled.div`
  background-color: #f8d7da;
  border: 1px solid #f5c6cb;
  padding: 15px;
  border-radius: 8px;
  color: #721c24;
  font-family: sans-serif;
  margin-bottom: 10px;
`;

function AlertStyled() {
  return (
    <StyledCard>
      <strong>Alert:</strong> Overdraft detected for client: Lito.
    </StyledCard>
  );
}
export default AlertStyled;
```

Method 4: Tailwind CSS

1. Create AlertTailwind.jsx.
2. Paste this code:

```
function AlertTailwind() {
  return (
    // Look at how much shorter the JavaScript is! All styling is in the className string.
    <div className="bg-blue-100 border border-blue-200 p-4 rounded-lg text-blue-800 font-sans mb-2">
```

```
    <strong>Info:</strong> New account created for client: Win.
   </div>
  );
}
export default AlertTailwind;
```

View the Comparison
Open your App.jsx, delete everything, and paste this to see all four methods side-by-side:

```
import AlertInline from './AlertInline';
import AlertModule from './AlertModule';
import AlertStyled from './AlertStyled';
import AlertTailwind from './AlertTailwind';

function App() {
  return (
   <div style={{ padding: '40px', maxWidth: '500px' }}>
    <h2>Styling Methods Comparison</h2>
    <AlertInline />
    <AlertModule />
    <AlertStyled />
    <AlertTailwind />
   </div>
  );
}

export default App;
```

React Router

React Router is the magic that transforms a single-page component into a full-fledged, multi-page application (like moving from a "Home" page to a "Client Details" page) *without* ever reloading the browser.

Installing and configuring React Router

Because React is just a UI library, it doesn't have a built-in way to handle multiple pages. We have to install the industry standard: **React Router DOM**.

Instead of asking the server for a new HTML page when a user clicks a link, React Router intercepts the click, looks at the URL, and instantly swaps out the components on the screen.

Installation Stop the local server (Ctrl + C) and run: **npm install react-router-dom** Then, start the server again (npm run dev).

The Configuration (The BrowserRouter) To make routing work, the *entire* app must be wrapped in a special component called BrowserRouter. (We usually do this in main.jsx, but for simplicity in our sandbox, we can do it in App.jsx).

Before using routes, you must wrap your app in a Router provider. This acts as the "brain" that listens to the browser's URL bar.

Configuration Examples:

```
// Example A: The Standard Setup (usually done in main.jsx)
import { BrowserRouter } from 'react-router-dom';
const Config1 = () => (
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// Example B: Running an app inside a sub-folder
// If your app lives at "www.bank.com/portal", you use the "basename" prop
const Config2 = () => (
  <BrowserRouter basename="/portal">
    <App />
  </BrowserRouter>
);

// Example C: The HashRouter (for legacy servers)
// Creates URLs like: localhost:5173/#/directory instead of localhost:5173/directory
import { HashRouter } from 'react-router-dom';
const Config3 = () => (
  <HashRouter>
    <App />
  </HashRouter>
);

// Example D: The Modern "Data Router" Setup (React Router v6.4+)
// Professional standard, but slightly more complex for beginners
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
const router = createBrowserRouter([
  { path: "/", element: <Home /> },
]);
const Config4 = () => <RouterProvider router={router} />;
```

Route, Link, NavLink

These are the three core building blocks of navigation.

- **<Routes> and <Route>:** This is your map. It tells React: "If the URL is /about, show the <About /> component."
- **<Link>:** This is React's version of the standard HTML <a href="..."> tag. **Never use <a> tags in React for internal links!** An <a> tag will force the page to reload, destroying your app's state. A <Link> updates the URL instantly and smoothly.
- **<NavLink>:** A smarter version of <Link>. It automatically knows if the user is currently on that page, allowing you to easily highlight the "Active" menu item in a navigation bar.

<Route> tells the app *what* to show. <Link> tells the app *where* to go. <NavLink> does the same thing as <Link>, but it knows when it is "active."

Example 1: The Main Controller
Your App.jsx is the "traffic controller." This is where you place the **Routes** and **Route** definitions.

src/App.jsx

```
import { Routes, Route } from 'react-router-dom';
import { Navbar } from './components/Navbar';

// Simple placeholder components for the demo
const Home = () => <h1>Home Page</h1>;
const About = () => <h1>About Page</h1>;
const Contact = () => <h1>Contact Page</h1>;

function App() {
  return (
    <div>
      {/* The Navbar stays here so it shows on ALL pages */}
      <Navbar />

      {/* Demo A: The Switcher */}
      <main style={{ padding: '20px' }}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="*" element={<h1>404 - Page Not Found</h1>} />
        </Routes>
      </main>
    </div>
  );
}
export default App;
```

Dynamic routes

- Imagine you have 500 banking clients. You don't want to create 500 separate routes (/client/risa, /client/chiz, etc.).
- Instead, we create **Dynamic Routes** using a colon :. If we create a route called /client/:id, React Router will capture whatever is typed in the URL and pass it to our component using a Hook called useParams().

Instead of creating 100 hard-coded routes for 100 clients, we use a colon : to create a "variable" in the URL. We then use the useParams hook to read that variable.

Dynamic Route Examples:
Dynamic routes allow you to use a single component to render many different pages (like user profiles or product pages) based on a variable in the URL.

The Route Definition (App.jsx): Use a colon (:) to define a URL parameter.

```
<Route path="/profile/:username" element={<UserProfile />} />
```

The Component (UserProfile.jsx): Use the useParams hook to "grab" that value from the URL.

```
import { useParams } from 'react-router-dom';

export function UserProfile() {
  const { username } = useParams(); // Matches the :username in the Route

  return (
    <div>
      <h2>Profile Page</h2>
      <p>Welcome, <strong>{username}</strong>!</p>
    </div>
  );
}
```

Try visiting /profile/instructor or /profile/student to see it change

Nested routes

- Nested routes are "pages inside of pages." Think of a dashboard: the top navigation bar and side menu stay exactly the same, but the middle content area changes when you click different settings.
- We achieve this using an <Outlet />. The Outlet acts as a "placeholder" where the child components will be rendered.

Nested routes allow you to keep parent layouts (like a sidebar) visible while swapping out the child components inside them. You define the parent route, and put child <Route> tags *inside* of it. Then, the parent uses an <Outlet /> tag as a placeholder for where the children should appear.

Nested Route Examples:

```
import { Routes, Route, Outlet, Link } from 'react-router-dom';

// Example A: Defining nested routes inside a parent Route tag
const Nest1 = () => (
  <Routes>
    <Route path="/dashboard" element={<DashboardLayout />}>
      {/* These URLs become /dashboard/reports and /dashboard/settings */}
      <Route path="reports" element={<h2>Reports View</h2>} />
      <Route path="settings" element={<h2>Settings View</h2>} />
    </Route>
  </Routes>
);

// Example B: The Parent Component using <Outlet />
function DashboardLayout() {
  return (
    <div style={{ display: 'flex' }}>
      <aside> Sidebar Menu Here </aside>
      <main>
        {/* The child component (Reports or Settings) will inject right here! */}
        <Outlet />
      </main>
    </div>
  );
}

// Example C: The "Index" Route (The Default Child)
// When the user goes to just "/dashboard", what child shows up? The Index!
const Nest3 = () => (
  <Routes>
    <Route path="/dashboard" element={<DashboardLayout />}>
      <Route index element={<h2>Welcome to Dashboard Dashboard!</h2>} />
      <Route path="reports" element={<h2>Reports View</h2>} />
    </Route>
  </Routes>
);

// Example D: Linking to nested routes (Relative Links)
// Notice we don't need a slash "/" at the beginning of "reports"
function Sidebar() {
```

```
  return (
    <nav>
      <Link to="reports">View Reports</Link>
      <Link to="settings">Edit Settings</Link>
    </nav>
  );
}
```

Another Nested Route Example:
Nested routes allow you to keep part of the UI visible (like a sidebar) while swapping out a sub-section of the page. You must use the <Outlet /> component to tell React Router where the child content should appear.

The Route Definition (App.jsx): Wrap child routes inside a parent route.

```
<Route path="/dashboard" element={<DashboardLayout />}>
  {/* These render inside the DashboardLayout's Outlet */}
  <Route path="stats" element={<Stats />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

The Layout Component (DashboardLayout.jsx):

```
import { Outlet, Link } from 'react-router-dom';

export function DashboardLayout() {
  return (
    <div style={{ display: 'flex' }}>
      <aside style={{ width: '200px', background: '#f0f0f0' }}>
        <nav>
          <Link to="stats">Stats</Link><br />
          <Link to="settings">Settings</Link>
        </nav>
      </aside>

      <main style={{ padding: '20px' }}>
        <h1>Dashboard</h1>
        {/* Child routes (Stats/Settings) appear HERE */}
        <Outlet />
      </main>
    </div>
  );
}
```

Hands-on:

Build a Multi-Page Banking App

Build a 4-file application. This will feature a navigation bar with active states, a home page, a directory, and a dynamic client details page.

Step 1: The Navigation Bar (Navbar.jsx)
This component uses NavLink to highlight whichever page the user is currently visiting.

1. Create **Navbar.jsx** in your src folder.
2. Paste this code:

```
import { NavLink } from 'react-router-dom';

function Navbar() {
  // A helper function to change styles if the link is active
  const checkActive = ({ isActive }) => {
    return {
      padding: '10px 20px',
      textDecoration: 'none',
      color: isActive ? 'white' : '#ccc',
      backgroundColor: isActive ? '#00529B' : 'transparent', // Blue if active
      borderRadius: '5px',
      fontWeight: 'bold'
    };
  };

  return (
    <nav style={{ display: 'flex', gap: '15px', backgroundColor: '#333', padding: '15px', marginBottom: '20px'
}}>
      <NavLink to="/" style={checkActive}>Home</NavLink>
      <NavLink to="/directory" style={checkActive}>Client Directory</NavLink>
    </nav>
  );
}

export default Navbar;
```

Step 2: The Static Pages (Home.jsx & Directory.jsx)
These are standard components that will act as our separate pages.

1. Create **Home.jsx**:

```
function Home() {
  return (
    <div>
      <h1>Welcome to Bangko Sentral Portal</h1>
      <p>Select "Client Directory" from the menu to manage accounts.</p>
```

```
  </div>
 );
}
export default Home;
```

2. Create **Directory.jsx**. Notice how we use <Link> to dynamically generate URLs for specific clients

```
import { Link } from 'react-router-dom';

function Directory() {
 const clients = ["Risa", "Chiz", "Imee", "Bong", "Pia"];

 return (
  <div>
   <h2>Client Directory</h2>
   <ul style={{ listStyle: 'none', padding: 0 }}>
    {clients.map(client => (
     <li key={client} style={{ margin: '10px 0' }}>
      {/* This creates a link to /client/Risa, /client/Chiz, etc. */}
      <Link
       to={` /client/${client}` }
       style={{ textDecoration: 'none', color: '#00529B', fontSize: '18px' }}
      >
       View Account: {client}
      </Link>
     </li>
    ))}
   </ul>
  </div>
 );
}
export default Directory;
```

Step 3: The Dynamic Page (ClientDetails.jsx)

This page catches the dynamic URL using useParams().

1. Create **ClientDetails.jsx**:

```
import { useParams, Link } from 'react-router-dom';

function ClientDetails() {
 // 1. We extract the "id" from the URL (e.g., /client/Risa -> id becomes "Risa")
 const { id } = useParams();

 return (
  <div style={{ border: '2px solid #00529B', padding: '20px', borderRadius: '8px', maxWidth: '400px' }}>
   <h2>Client Profile: {id}</h2>
   <p><strong>Status:</strong> Active</p>
```

```
    <p><strong>Account Type:</strong> Premium Savings</p>
    <hr />
    {/* A link to go back to the directory */}
    <Link to="/directory" style={{ display: 'inline-block', marginTop: '15px' }}>
      ← Back to Directory
    </Link>
   </div>
  );
}

export default ClientDetails;
```

Step 4: The Router Setup (App.jsx)

Finally, we tie it all together. This is the "Map" of our application.

1. Open **App.jsx**, delete everything, and paste this:

```
// 1. Import all the routing tools
import { BrowserRouter, Routes, Route } from 'react-router-dom';

// 2. Import our page components
import Navbar from './Navbar';
import Home from './Home';
import Directory from './Directory';
import ClientDetails from './ClientDetails';

function App() {
  return (
   // The BrowserRouter must wrap everything!
   <BrowserRouter>
    <div style={{ fontFamily: 'sans-serif', padding: '20px' }}>

     {/* The Navbar stays OUTSIDE the <Routes> so it always shows on every page */}
     <Navbar />

     {/* The <Routes> act as the swapping area for our pages */}
     <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/directory" element={<Directory />} />

      {/* This is the DYNAMIC route. Note the colon ":" */}
      <Route path="/client/:id" element={<ClientDetails />} />

      {/* A Catch-All route for 404 Not Found pages */}
      <Route path="*" element={<h2>404 - Page Not Found</h2>} />
     </Routes>
```

```
    </div>
   </BrowserRouter>
  );
}

export default App;
```

Test

1. Save all 5 files.
2. Go to your browser. Notice the **Navbar** at the top. The "Home" link will be highlighted blue because we used NavLink.
3. Click "Client Directory". Watch the URL change to localhost:5173/directory. The screen instantly swaps to the list of senators, and the Navbar active highlight moves.
4. Click on "View Account: Imee". The URL changes to /client/Imee. React Router intercepts this, mounts the ClientDetails component, and useParams() grabs "Imee" to display her specific profile.

Advanced Hooks

useContext for global state

Imagine you have a "Parent" component, and a "Great-Great-Grandchild" component. To pass data to the grandchild using normal props, you have to pass it through every single component in between. This is called **"Prop Drilling"** and it makes code very messy.

useContext is like setting up a global PA system or a teleportation portal. You broadcast the data at the very top of your app, and *any* component, no matter how deep, can "listen in" and grab that data instantly.

Examples of useContext Elements:
Example 1: The Theme Switcher (Light/Dark Mode)
This splits the Context creation and consumption into separate files, which is how you will build apps in the real world.

1. Create the component that uses the context:

- **File Placement:** src/ThemeButton.jsx

```
import React, { useContext } from 'react';
import { ThemeContext } from './App'; // Import the context we export from App.js

function ThemeButton() {
  const { theme, toggleTheme } = useContext(ThemeContext);

  return (
    <button
      onClick={toggleTheme}
      style={{
```

```
      background: theme === 'light' ? '#fff' : '#333',
      color: theme === 'light' ? '#000' : '#fff',
      padding: '10px 20px',
      cursor: 'pointer'
    }}
   >
    Current Theme: {theme} (Click to toggle)
   </button>
  );
}

export default ThemeButton;
```

2. Set up the Provider in your main file:

- **File Placement:** src/App.jsx

```
import React, { useState, createContext } from 'react';
import ThemeButton from './ThemeButton';

// 1. Create and EXPORT the Context so other files can use it
export const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme(theme === 'light' ? 'dark' : 'light');

  return (
   // 2. Wrap the children in the Provider
   <ThemeContext.Provider value={{ theme, toggleTheme }}>
    <div style={{ padding: '50px', background: theme === 'light' ? '#eee' : '#111', height: '100vh' }}>
     <ThemeButton />
    </div>
   </ThemeContext.Provider>
  );
}

export default App;
```

Click the button. You will see both the button's styling and the entire background (controlled by App.jsx) change instantly.

Example 2: Shopping Cart Counter
This demonstrates how a button deep in your app can update a counter that lives in your navigation bar.

1. Create the child components:

- **File Placement:** src/StoreComponents.jsx

```
import React, { useContext } from 'react';
import { CartContext } from './App';

// Component 1: The Navbar that reads the cart count
export function Navbar() {
  const { cartCount } = useContext(CartContext);
  return (
    <nav style={{ background: '#2196F3', color: 'white', padding: '15px' }}>
      <h2>My Store 🛒 (Items in Cart: {cartCount})</h2>
    </nav>
  );
}

// Component 2: The Product button that updates the cart count
export function Product() {
  const { addToCart } = useContext(CartContext);
  return (
    <div style={{ border: '1px solid gray', padding: '20px', margin: '20px', width: '150px' }}>
      <h3>Cool Shoes</h3>
      <button onClick={addToCart}>Add to Cart</button>
    </div>
  );
}
```

2. Set up the Provider:

- **File Placement:** src/App.jsx

```
import React, { useState, createContext } from 'react';
import { Navbar, Product } from './StoreComponents';

export const CartContext = createContext();

function App() {
  const [cartCount, setCartCount] = useState(0);
  const addToCart = () => setCartCount(prev => prev + 1);

  return (
    <CartContext.Provider value={{ cartCount, addToCart }}>
      <Navbar />
      <Product />
      <Product /> {/* You can render multiple, they all share the same context! */}
```

```
    </CartContext.Provider>
  );
}

export default App;
```

Click "Add to Cart" on any of the product boxes. The cart number in the blue navigation bar will increase.

Example 3: Global User Authentication
This mimics logging in and conditionally rendering a dashboard versus a login screen based on global state.

1. Create the dashboard component:

- **File Placement:** src/Dashboard.jsx

```
import React, { useContext } from 'react';
import { AuthContext } from './App';

function Dashboard() {
  const { user, logout } = useContext(AuthContext);

  // If there's no user, show nothing (or a warning)
  if (!user) return <p>Please log in.</p>;

  return (
    <div style={{ padding: '20px', border: '2px dashed green' }}>
      <h1>Welcome to your Dashboard, {user.name}!</h1>
      <p>Your email: {user.email}</p>
      <button onClick={logout}>Log Out</button>
    </div>
  );
}

export default Dashboard;
```

2. Set up the Provider and Login flow:

- **File Placement:** src/App.jsx

```
import React, { useState, createContext } from 'react';
import Dashboard from './Dashboard';

export const AuthContext = createContext();

function App() {
  const [user, setUser] = useState(null); // null means not logged in

  const login = () => {
```

```
  // Simulating fetching a user from a database
  setUser({ name: 'Alex', email: 'alex@example.com' });
 };

 const logout = () => setUser(null);

 return (
  <AuthContext.Provider value={{ user, logout }}>
   <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
    {/* Conditional rendering based on Context state */}
    {!user ? (
     <div>
      <h2>You are logged out.</h2>
      <button onClick={login}>Log In</button>
     </div>
    ) : (
     <Dashboard />
    )}
   </div>
  </AuthContext.Provider>
 );
}

export default App;
```

The screen will initially say you are logged out. Click "Log In". The entire view will swap to the Dashboard component, which automatically knows the user's name and email.

useReducer for complex state

useState is great for simple things like a name or a toggle. But what if your state is a massive object with 10 different properties that update based on complex rules (like a shopping cart or a bank account)?

useReducer is the heavy-duty version of useState. Instead of just updating the state directly, you send an "Action" (like a set of instructions) to a "Reducer Function." The Reducer acts as a strict accountant, looking at the instructions and calculating the new state.

Examples of useReducer Elements:

Example 1: The Classic Counter (The Basics)
This is the standard "Hello World" of useReducer. It shows the core pattern: you dispatch an **action**, the **reducer** intercepts it, and updates the **state**.

1. Create the Counter component and its Reducer:

- **File Placement:** src/Counter.jsx

```
import React, { useReducer } from 'react';

// 1. Define the reducer function OUTSIDE the component.
// It takes the current state and an action, and returns the NEW state.
function reducer(state, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    case 'RESET':
      return { count: 0 };
    default:
      return state;
  }
}

function Counter() {
  // 2. Initialize useReducer with the reducer function and the starting state
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div style={{ padding: '20px', border: '1px solid #ccc', width: '250px' }}>
      <h2>Count: {state.count}</h2>

      {/* 3. Dispatch actions to trigger state changes */}
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>+</button>
      <button onClick={() => dispatch({ type: 'DECREMENT' })} style={{ margin: '0 10px' }}>-</button>
      <button onClick={() => dispatch({ type: 'RESET' })}>Reset</button>
    </div>
  );
}

export default Counter;
```

2. Render it in your main file:

- **File Placement:** src/App.jsx

```
import React from 'react';
import Counter from './Counter';

function App() {
  return (
    <div style={{ padding: '40px', fontFamily: 'sans-serif' }}>
```

```
    <h1>useReducer Demo 1</h1>
    <Counter />
   </div>
  );
}

export default App;
```

Click the +, -, and Reset buttons. Notice how the component doesn't directly update the count; instead, it simply tells the reducer *what happened* (the action), and the reducer handles the math.

Example 2: A Simple Todo List (Handling Arrays and Payloads)
This demonstrates where useReducer actually outshines useState: managing an array of objects where you need to pass extra data (a payload) along with your action.

1. Create the Todo component and its Reducer:

- **File Placement:** src/TodoList.jsx

```
import React, { useReducer, useState } from 'react';

// 1. The reducer handles adding new items and toggling their status
function todoReducer(todos, action) {
  switch (action.type) {
    case 'ADD_TODO':
      return [
        ...todos,
        { id: Date.now(), text: action.payload, completed: false }
      ];
    case 'TOGGLE_TODO':
      return todos.map(todo =>
        todo.id === action.payload ? { ...todo, completed: !todo.completed } : todo
      );
    default:
      return todos;
  }
}

function TodoList() {
  const [todos, dispatch] = useReducer(todoReducer, []);
  const [input, setInput] = useState(''); // We still use useState for simple form inputs!

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!input.trim()) return;

    // Dispatch the action WITH a payload (the task text)
```

```
  dispatch({ type: 'ADD_TODO', payload: input });
  setInput('');
 };

 return (
  <div style={{ padding: '20px', maxWidth: '300px' }}>
   <form onSubmit={handleSubmit} style={{ marginBottom: '20px' }}>
    <input
     value={input}
     onChange={(e) => setInput(e.target.value)}
     placeholder="Add a task..."
    />
    <button type="submit">Add</button>
   </form>

   <ul style={{ listStyle: 'none', padding: 0 }}>
    {todos.map(todo => (
     <li
      key={todo.id}
      onClick={() => dispatch({ type: 'TOGGLE_TODO', payload: todo.id })}
      style={{
       textDecoration: todo.completed ? 'line-through' : 'none',
       cursor: 'pointer',
       padding: '8px 0',
       borderBottom: '1px solid #eee'
      }}
     >
      {todo.completed ? '✅' : '⬜'} {todo.text}
     </li>
    ))}
   </ul>
  </div>
 );
}

export default TodoList;
```

2. Render it in your main file:

- **File Placement:** src/App.jsx

```
import React from 'react';
import TodoList from './TodoList';

function App() {
 return (
  <div style={{ padding: '40px', fontFamily: 'sans-serif' }}>
```

```
    <h1>useReducer Demo 2</h1>
    <TodoList />
   </div>
  );
}

export default App;
```

Type a task into the input box and press Enter (or click Add). It will appear in the list below. Click directly on the task text; it will instantly cross out and show a green checkmark, demonstrating the toggle functionality.

Custom hooks

A Custom Hook is just a normal JavaScript function that starts with the word use (e.g., useWindowSize, useFetch). If you find yourself writing the exact same useState and useEffect logic in multiple files, you can extract it into your own custom hook to reuse it anywhere.

Demo 1: The useToggle Hook (Simplest Example)
Instead of writing a complex toggle function every time you want to show/hide a modal or switch a boolean value, this hook handles the boilerplate for you.

1. Create the Custom Hook:

- **File Placement:** src/hooks/useToggle.js *(Note: It's good practice to keep hooks in their own folder)*

```
import { useState } from 'react';

export function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  // The toggle function switches the state to the opposite of what it currently is
  const toggle = () => {
    setValue((currentValue) => !currentValue);
  };

  // We return the state and the toggle function in an array (just like useState does)
  return [value, toggle];
}
```

2. Use it in your component:

- **File Placement:** src/App.jsx

```
import React from 'react';
import { useToggle } from './hooks/useToggle';

function App() {
  // We use our custom hook exactly like a built-in hook!
```

```
  const [isModalOpen, toggleModal] = useToggle(false);

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h1>useToggle Demo</h1>
      <button onClick={toggleModal}>
        {isModalOpen ? 'Close Modal' : 'Open Modal'}
      </button>

      {isModalOpen && (
        <div style={{ marginTop: '20px', padding: '20px', background: '#e0e0e0' }}>
          <p>Here is the secret modal content!</p>
        </div>
      )}
    </div>
  );
}

export default App;
```

Click the button. The modal will appear and disappear. You can now use useToggle anywhere in your app without rewriting the state-flipping logic.

Example 2: The useWindowSize Hook (Handling Events)
This hook abstracts away the messy useEffect logic needed to listen to browser window resize events, giving your component clean access to the current width and height.

1. Create the Custom Hook:

- **File Placement:** src/hooks/useWindowSize.js

```
import { useState, useEffect } from 'react';

export function useWindowSize() {
  const [windowSize, setWindowSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    // The function that updates state when the window resizes
    function handleResize() {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    }
```

```
  // Add event listener
  window.addEventListener('resize', handleResize);

  // Cleanup function to remove listener when component unmounts
  return () => window.removeEventListener('resize', handleResize);
 }, []); // Empty array ensures this effect only runs on mount

 return windowSize;
}
```

2. Use it in your component:

- **File Placement:** src/App.jsx

```
import React from 'react';
import { useWindowSize } from './hooks/useWindowSize';

function App() {
 const { width, height } = useWindowSize();

 return (
  <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
   <h1>useWindowSize Demo</h1>
   <p>Resize your browser window to see the values update in real-time:</p>

   <div style={{ padding: '20px', border: '2px dashed blue', display: 'inline-block' }}>
    <strong>Width:</strong> {width}px <br />
    <strong>Height:</strong> {height}px
   </div>

   {width < 600 && <p style={{ color: 'red' }}>Mobile view activated!</p>}
  </div>
 );
}

export default App;
```

Grab the edge of your browser window and resize it. You will see the width and height numbers update instantly, and a red warning message will appear if you shrink the width below 600 pixels.

Example 3: The useLocalStorage Hook (Syncing state with the browser)
This is an incredibly useful hook. It acts exactly like useState, but it also automatically saves the value to the browser's Local Storage. If the user refreshes the page, their data is still there!

1. Create the Custom Hook:

- **File Placement:** src/hooks/useLocalStorage.js

```
import { useState, useEffect } from 'react';

export function useLocalStorage(key, initialValue) {
  // 1. Initialize state by checking local storage first
  const [value, setValue] = useState(() => {
    const savedValue = localStorage.getItem(key);
    if (savedValue) return JSON.parse(savedValue);
    return initialValue;
  });

  // 2. Whenever the key or the value changes, update local storage
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}
```

2. Use it in your component:

- **File Placement:** src/App.jsx

```
import React from 'react';
import { useLocalStorage } from './hooks/useLocalStorage';

function App() {
  // Notice how similar this looks to standard useState!
  // It takes a key ('username') and an initial value ('')
  const [name, setName] = useLocalStorage('username', '');

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h1>useLocalStorage Demo</h1>
      <p>Type your name below, then refresh the browser page.</p>

      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter your name"
        style={{ padding: '5px', fontSize: '16px' }}
```

```
    />

    {name && <p>Hello, {name}! Your data is safely stored.</p>}
   </div>
  );
}

export default App;
```

Type your name into the input field. Then, physically refresh your browser page (Ctrl+R or Cmd+R). Normally, React state resets on refresh, but because of our custom hook, your name will still be in the input box

Hands-on:

Theme switcher with context

Build a global "Dark Mode / Light Mode" switch. Wrap the app in a Context Provider so any component can instantly know the current theme and change it.

Step 1: Create the Context File
In your src folder, create **ThemeContext.jsx**:

```
import { createContext, useState } from 'react';

// 1. Create the empty Context Portal
export const ThemeContext = createContext();

// 2. Create a "Wrapper" component that holds the state and broadcasts it
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light"); // Default is light mode

  const toggleTheme = () => {
    setTheme(theme === "light" ? "dark" : "light");
  };

  return (
    // We broadcast an object containing the current theme AND the toggle function
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Step 2: Consume the Context in a Component

Create **Dashboard.jsx**:

```
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

function Dashboard() {
  // 3. We reach into the portal and grab the theme data! No props needed.
  const { theme, toggleTheme } = useContext(ThemeContext);

  // Set styles based on the global theme
  const dashboardStyle = {
    backgroundColor: theme === "light" ? "#ffffff" : "#333333",
    color: theme === "light" ? "#000000" : "#ffffff",
    padding: '20px',
    height: '100vh',
    transition: 'all 0.3s ease'
  };

  return (
    <div style={dashboardStyle}>
      <h2>Global Settings Portal</h2>
      <p>Current Theme: {theme.toUpperCase()}</p>
      <button onClick={toggleTheme} style={{ padding: '10px' }}>
        Switch to {theme === "light" ? "Dark" : "Light"} Mode
      </button>
    </div>
  );
}

export default Dashboard;
```

Step 3: Test in App.jsx

Open **App.jsx**, delete everything, and wrap your app:

```
import { ThemeProvider } from './ThemeContext';
import Dashboard from './Dashboard';

function App() {
  return (
    // Everything inside ThemeProvider now has access to the global state!
    <ThemeProvider>
      <Dashboard />
    </ThemeProvider>
  );
}
export default App;
```

Build a custom data-fetching hook

Fetching data from APIs always requires useState (for the data, loading state, and error state) and useEffect (to make the actual call). Let's bundle all that repetitive code into one elegant Custom Hook.

Step 1: Create the Custom Hook
Create a file named **useFetch.jsx**:

```
import { useState, useEffect } from 'react';

// Custom hooks must start with "use"
function useFetch(url) {
  // Set up the 3 standard states for any API call
  const [data, setData] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // Reset states when URL changes
    setIsLoading(true);
    setError(null);

    // The actual fetch call
    fetch(url)
      .then(response => {
        if (!response.ok) throw new Error("Failed to fetch data.");
        return response.json();
      })
      .then(jsonData => {
        setData(jsonData);
        setIsLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setIsLoading(false);
      });
  }, [url]); // Dependency array: re-run if the URL changes

  // Return the data exactly like how useState returns an array
  return { data, isLoading, error };
}

export default useFetch;
```

Step 2: Test the Hook in App.jsx

We will test this using a free, fake API for placeholder data. Open **App.jsx** and replace it with:

```
import useFetch from './useFetch';

function App() {
  // We call our custom hook with a URL, and instantly get back all 3 states!
  const { data, isLoading, error } = useFetch("https://jsonplaceholder.typicode.com/users");

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h2>External Client Database</h2>

      {/* Conditional Rendering based on our Custom Hook's states */}
      {isLoading && <p style={{ color: 'blue' }}>Loading records from server...</p>}

      {error && <p style={{ color: 'red' }}>Error: {error}</p>}

      {data && (
        <ul>
          {/* We map over the fetched data */}
          {data.slice(0, 5).map(user => (
            <li key={user.id} style={{ margin: '10px 0', borderBottom: '1px solid #ccc' }}>
              <strong>{user.name}</strong> - {user.email}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
export default App;
```

Save the files in your Windsurf editor and view the browser. You will see a quick blue "Loading..." message, followed by a dynamically fetched list of external users, all powered by one cleanly abstracted custom hook.

Data Fetching

Fetch API and Axios

To get data from the internet, JavaScript needs a tool to send the HTTP request.

- **Fetch API:** This is built directly into every modern browser. You don't need to install anything. However, it requires an extra step to convert the data into JSON, and it doesn't automatically throw an error if the server returns a 404 (Not Found).

- **Axios:** This is the industry-standard third-party library (npm install axios). Professional developers love it because it automatically converts data to JSON, handles errors better, and requires less typing.

The Fetch API (Built-in)
You don't need to install anything to use Fetch; it is built directly into modern browsers.

Fetch Demo 1: Basic GET Request
This is the standard way to retrieve data and display it when a component loads.

- **File Placement:** src/FetchGet.jsx

```
import React, { useState, useEffect } from 'react';

function FetchGet() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    // 1. Call the API
    fetch('https://jsonplaceholder.typicode.com/users')
      // 2. Convert the raw response to JSON
      .then(response => response.json())
      // 3. Save the data to state
      .then(data => setUsers(data));
  }, []);

  return (
    <div style={{ padding: '20px' }}>
      <h2>Fetch GET Demo: User List</h2>
      <ul>
        {users.slice(0, 3).map(user => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
  );
}

export default FetchGet;
```

Import and render <FetchGet /> in your src/App.jsx. When the page loads, you will see a list of three user names fetched from the fake API.

Fetch Demo 2: POST Request (Sending Data)
When you submit a form, you need to send data *to* the server. Fetch requires you to manually stringify your data and set the headers.

- **File Placement:** src/FetchPost.jsx

```
import React, { useState } from 'react';

function FetchPost() {
  const [status, setStatus] = useState('Idle');

  const handlePost = () => {
    setStatus('Sending...');

    fetch('https://jsonplaceholder.typicode.com/posts', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'My New Post', body: 'Hello world!', userId: 1 })
    })
      .then(response => response.json())
      .then(data => setStatus(` Success! New Post ID: ${data.id}` ));
  };

  return (
    <div style={{ padding: '20px' }}>
      <h2>Fetch POST Demo</h2>
      <button onClick={handlePost}>Create Post</button>
      <p>Status: {status}</p>
    </div>
  );
}

export default FetchPost;
```

Import and render <FetchPost /> in App.jsx. Click "Create Post". The status will briefly say "Sending..." and then return a success message with the newly generated ID from the server.

Fetch Demo 3: Proper Error Handling
Fetch *only* rejects a promise if the network completely fails. It does **not** reject for 404 (Not Found) or 500 (Server Error). You have to check response.ok manually.

- **File Placement:** src/FetchError.jsx

```
import React, { useState } from 'react';

function FetchError() {
  const [error, setError] = useState(null);

  const fetchBadUrl = async () => {
```

```
  try {
    setError(null);
    // Notice the typo in the URL ('userzz')
    const response = await fetch('https://jsonplaceholder.typicode.com/userzz');

    // With Fetch, you MUST manually check if the response was successful
    if (!response.ok) {
      throw new Error(` HTTP error! Status: ${response.status}` );
    }

    const data = await response.json();
    console.log(data);
  } catch (err) {
    setError(err.message);
  }
 };

 return (
  <div style={{ padding: '20px' }}>
    <h2>Fetch Error Handling Demo</h2>
    <button onClick={fetchBadUrl}>Fetch Bad Data</button>
    {error && <p style={{ color: 'red' }}>Caught Error: {error}</p>}
  </div>
 );
}

export default FetchError;
```

Import and render <FetchError /> in App.jsx. Click the button. Because the URL is invalid, it will catch the 404 error and display it in red text on your screen.

Part 2: Axios
To use these demos, you must first open your terminal and run: **npm install axios**

Axios Demo 1: Basic GET Request
Notice how we don't need a .then(res => res.json()) step anymore. Axios automatically converts the response to JSON and puts it inside a data property.

- **File Placement:** src/AxiosGet.jsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

function AxiosGet() {
 const [posts, setPosts] = useState([]);

 useEffect(() => {
```

```
    // Axios automatically parses the JSON for you!
    axios.get('https://jsonplaceholder.typicode.com/posts')
      .then(response => setPosts(response.data));
  }, []);

  return (
    <div style={{ padding: '20px' }}>
      <h2>Axios GET Demo: Posts</h2>
      <ul>
        {posts.slice(0, 2).map(post => (
          <li key={post.id}><strong>{post.title}</strong></li>
        ))}
      </ul>
    </div>
  );
}

export default AxiosGet;
```

Import and render <AxiosGet /> in App.jsx. When the component loads, it will fetch and display the titles of two posts.

Axios Demo 2: POST Request (Sending Data)
Axios makes POST requests much cleaner. You don't need to manually stringify the body or set the Content-Type header; it does all of that automatically.

- **File Placement:** src/AxiosPost.jsx

```
import React, { useState } from 'react';
import axios from 'axios';

function AxiosPost() {
  const [responseMsg, setResponseMsg] = useState('');

  const handlePost = async () => {
    try {
      // Just pass the JS object directly as the second argument
      const response = await axios.post('https://jsonplaceholder.typicode.com/posts', {
        title: 'Axios is easy',
        body: 'No stringify needed!',
        userId: 2
      });
      setResponseMsg(`Saved with ID: ${response.data.id}`);
    } catch (error) {
      setResponseMsg('An error occurred');
    }
  };
```

```
  return (
    <div style={{ padding: '20px' }}>
      <h2>Axios POST Demo</h2>
      <button onClick={handlePost}>Send Data with Axios</button>
      <p>{responseMsg}</p>
    </div>
  );
}

export default AxiosPost;
```

Import and render <AxiosPost /> in App.jsx. Click the button to trigger the POST request and see the resulting ID from the fake server.

Axios Demo 3: Creating an Axios Instance (Base URLs)
If your app makes calls to https://api.mywebsite.com everywhere, typing that full URL every time gets annoying. Axios lets you create a reusable "instance" with a pre-set base URL.

- **File Placement:** src/api.js *(Create a standard JS file for this)*

```
import axios from 'axios';

// 1. Create a configured instance
export const api = axios.create({
  baseURL: 'https://jsonplaceholder.typicode.com',
  timeout: 5000, // Abort if it takes longer than 5 seconds
});
```

- **File Placement:** src/AxiosInstanceDemo.jsx

```
import React, { useState } from 'react';
import { api } from './api'; // Import your custom instance

function AxiosInstanceDemo() {
  const [data, setData] = useState(null);

  const fetchData = async () => {
    // 2. You only need to type the endpoint path now!
    const response = await api.get('/todos/1');
    setData(response.data.title);
  };

  return (
    <div style={{ padding: '20px' }}>
      <h2>Axios Instance Demo</h2>
      <button onClick={fetchData}>Get Todo #1</button>
      {data && <p>Todo: {data}</p>}
```

```
  </div>
 );
}

export default AxiosInstanceDemo;
```

Import and render <AxiosInstanceDemo /> in App.jsx. Click the button. It uses the custom api instance to cleanly fetch /todos/1 using the base URL you configured in api.js.

Async/await in React

Network requests take time. JavaScript doesn't like to wait; it wants to keep running code. We use async and await to force JavaScript to pause and wait for the server to reply before moving to the next line of code.

You **cannot** make the useEffect function itself async. You must create an async function *inside* the effect, and then call it.

Examples of Async/Await:
Example 1: The useEffect Fetch Pattern
A very common mistake in React is trying to make the useEffect callback itself async (e.g., useEffect(async () => {...})). React forbids this. Instead, you must define an async function *inside* the effect and then immediately call it.

- **File Placement:** src/AsyncFetch.jsx

```
import React, { useState, useEffect } from 'react';

function AsyncFetch() {
 const [user, setUser] = useState(null);

 useEffect(() => {
  // 1. Define the async function INSIDE the useEffect
  const fetchUser = async () => {
   // 2. Use 'await' to pause this function until the fetch finishes
   const response = await fetch('https://jsonplaceholder.typicode.com/users/1');

   // 3. Use 'await' again to pause while parsing the JSON
   const userData = await response.json();

   // 4. Finally, update the state
   setUser(userData);
  };

  // 5. Call the function we just defined
  fetchUser();
 }, []);
```

```
  if (!user) return <div style={{ padding: '20px' }}>Loading user...</div>;

  return (
    <div style={{ padding: '20px' }}>
      <h2>Async/Await Demo 1</h2>
      <p><strong>Name:</strong> {user.name}</p>
      <p><strong>Email:</strong> {user.email}</p>
    </div>
  );
}

export default AsyncFetch;
```

Import and render <AsyncFetch /> in your src/App.jsx. When the page loads, it will briefly say "Loading user..." before cleanly rendering the fetched data.

Example 2: Clean Error Handling with try / catch

When using standard Promises, you use .catch() to handle errors. With async / await, you wrap your code in a standard JavaScript try...catch block. This makes handling complex logic much easier to read.

- **File Placement:** src/AsyncErrorHandling.jsx

```
import React, { useState } from 'react';

function AsyncErrorHandling() {
  const [status, setStatus] = useState('Waiting for action');

  const handleFetch = async () => {
    setStatus('Loading...');

    // 1. TRY to execute this block of code
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/invalid-url-here');

      if (!response.ok) {
        // Manually throw an error if the status is 404, 500, etc.
        throw new Error(`Server responded with a ${response.status} error.`);
      }

      const data = await response.json();
      setStatus(`Success! Data: ${data.title}`);

    // 2. CATCH any errors that get thrown in the 'try' block
    } catch (error) {
      setStatus(`Failed: ${error.message}`);
    }
  };
```

```
  return (
    <div style={{ padding: '20px' }}>
      <h2>Async/Await Demo 2 (Errors)</h2>
      <button onClick={handleFetch}>Fetch Broken URL</button>
      <p style={{ marginTop: '10px', fontWeight: 'bold' }}>Status: {status}</p>
    </div>
  );
}

export default AsyncErrorHandling;
```

Import and render <AsyncErrorHandling /> in App.jsx. Click the button. The status will briefly change to "Loading..." and then safely catch the 404 error, updating the screen with the failure message instead of crashing your app.

Example 3: The "Sleep" Utility (Simulating Delays)
This demo perfectly illustrates how await literally pauses the execution of a function. We will create a custom delay function to simulate a multi-step process (like connecting to a database, processing a payment, etc.).

- **File Placement:** src/AsyncSteps.jsx

```
import React, { useState } from 'react';

// A simple utility function that returns a Promise that resolves after X milliseconds
const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms));

function AsyncSteps() {
  const [logs, setLogs] = useState([]);
  const [isProcessing, setIsProcessing] = useState(false);

  const addLog = (message) => {
    setLogs(prevLogs => [...prevLogs, message]);
  };

  const startProcess = async () => {
    setIsProcessing(true);
    setLogs([]); // Clear previous logs

    addLog('1. Initializing process...');
    await delay(1000); // Pause this function for 1 second

    addLog('2. Verifying credentials...');
    await delay(1500); // Pause for 1.5 seconds

    addLog('3. Securely saving data...');
```

```
  await delay(1000); // Pause for 1 second

  addLog('✅ Process complete!');
  setIsProcessing(false);
 };

 return (
  <div style={{ padding: '20px' }}>
   <h2>Async/Await Demo 3 (Delays)</h2>
   <button onClick={startProcess} disabled={isProcessing}>
    {isProcessing ? 'Processing...' : 'Start Multi-Step Process'}
   </button>

   <ul style={{ marginTop: '20px', fontFamily: 'monospace' }}>
    {logs.map((log, index) => (
     <li key={index} style={{ padding: '5px 0' }}>{log}</li>
    ))}
   </ul>
  </div>
 );
}

export default AsyncSteps;
```

Import and render <AsyncSteps /> in App.jsx. Click the "Start Multi-Step Process" button. You will see the UI update step-by-step, waiting the exact specified amount of time between each log, proving exactly how await handles asynchronous pauses without freezing the browser!

Handling loading and error states

Because fetching takes time and the internet is unreliable, your UI needs three distinct states:

1. **Loading:** Is the data currently traveling through the wires? (Show a spinner).
2. **Error:** Did the server crash or the wifi drop? (Show an error message).
3. **Success:** Do we have the data? (Show the list).

Examples of State Management:

```
// Example A: Setting up the "Holy Trinity" of fetching states
const [data, setData] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);

// Example B: The Try/Catch Block
// This is how we trap errors when using async/await
const fetchData = async () => {
 try {
```

```
    const res = await fetch('url');
    if (!res.ok) throw new Error("Server rejected request");
    // Success path
  } catch (err) {
    setError(err.message); // Error path
  } finally {
    setIsLoading(false); // This runs no matter what happens!
  }
};

// Example C: Early Return Pattern for UI
// The cleanest way to show loading/errors before returning the main component.
function DataView() {
  if (isLoading) return <p>Loading Senator files...</p>;
  if (error) return <p style={{color: 'red'}}>Error: {error}</p>;
  return <div>{data.name}</div>;
}

// Example D: Inline Conditional Rendering
const Ex4 = () => (
  <div>
    {isLoading && <span className="spinner">⏳</span>}
    {!isLoading && !error && <DataTable rows={data} />}
  </div>
);
```

Hands-on:

Fetch data from a public API and Display with pagination

Use the free **JSONPlaceholder** API to simulate fetching a large list of official public notices. Because the list is 100 items long, we will build a **Pagination** system to only show 5 items per page!

Step 1: Create the Component

1. In your src folder, create a new file named **PublicNotices.jsx**.
2. Paste this code. Have your students pay close attention to the math logic for slicing the array!

```
import { useState, useEffect } from 'react';

function PublicNotices() {
  // 1. The Fetching States
  const [notices, setNotices] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  // 2. The Pagination States
```

```
const [currentPage, setCurrentPage] = useState(1);
const recordsPerPage = 5; // We only want to show 5 items at a time

// 3. The Effect: Fetch the data exactly once when the component loads
useEffect(() => {
  const fetchNotices = async () => {
    try {
      setIsLoading(true);
      // We are fetching 100 fake posts from a public testing API
      const response = await fetch('https://jsonplaceholder.typicode.com/posts');

      if (!response.ok) throw new Error("Failed to fetch public notices.");

      const data = await response.json();
      setNotices(data); // Save all 100 items to state
    } catch (err) {
      setError(err.message);
    } finally {
      setIsLoading(false); // Stop the loading spinner
    }
  };

  fetchNotices();
}, []); // Empty array means this only runs once!

// 4. Pagination Math Calculations
// If we are on Page 2, and showing 5 per page:
// indexOfLastRecord = 2 * 5 = 10
// indexOfFirstRecord = 10 - 5 = 5
// We then "slice" the array to only grab items 5 through 9.
const indexOfLastRecord = currentPage * recordsPerPage;
const indexOfFirstRecord = indexOfLastRecord - recordsPerPage;
const currentRecords = notices.slice(indexOfFirstRecord, indexOfLastRecord);

// 5. Button Handlers
const handleNext = () => setCurrentPage(prev => prev + 1);
const handlePrev = () => setCurrentPage(prev => prev - 1);

// 6. Conditional UI Rendering
if (isLoading) return <div style={{ padding: '20px' }}> Connecting to database...</div>;
if (error) return <div style={{ color: 'red', padding: '20px' }}> {error}</div>;

return (
  <div style={{ padding: '20px', maxWidth: '600px', fontFamily: 'sans-serif' }}>
    <h2>National Public Notices</h2>
```

```
    <p style={{ color: 'gray' }}>Displaying Page {currentPage}</p>

    {/* Map through ONLY the sliced 5 records for this page */}
    <ul style={{ listStyle: 'none', padding: 0 }}>
     {currentRecords.map(notice => (
       <li
        key={notice.id}
        style={{ border: '1px solid #ccc', padding: '15px', marginBottom: '10px', borderRadius: '5px' }}
       >
        <h4 style={{ margin: '0 0 10px 0', color: '#00529B', textTransform: 'capitalize' }}>
         Notice #{notice.id}: {notice.title.substring(0, 30)}...
        </h4>
        <p style={{ margin: 0, color: '#333' }}>
         {notice.body.substring(0, 80)}...
        </p>
       </li>
     ))}
    </ul>

    {/* Pagination Controls */}
    <div style={{ display: 'flex', justifyContent: 'space-between', marginTop: '20px' }}>
     <button
      onClick={handlePrev}
      disabled={currentPage === 1} // Disable if we are on page 1
      style={{ padding: '10px 20px', cursor: currentPage === 1 ? 'not-allowed' : 'pointer' }}
     >
      ← Previous
     </button>

     <button
      onClick={handleNext}
      // Disable if we've reached the end of the array
      disabled={indexOfLastRecord >= notices.length}
      style={{ padding: '10px 20px', cursor: indexOfLastRecord >= notices.length ? 'not-allowed' : 'pointer'
}}
     >
      Next →
     </button>
    </div>
   </div>
  );
}

export default PublicNotices;
```

Step 2: Test it in App.jsx

1. Open your **App.jsx** file in Windsurf.
2. Delete the contents and paste this to render our new component:

```
import PublicNotices from './PublicNotices';

function App() {
  return (
    <div style={{ backgroundColor: '#f4f4f9', minHeight: '100vh', padding: '20px' }}>
      <PublicNotices />
    </div>
  );
}

export default App;
```

Save both files and look at your browser.

1. You should briefly see the "Connecting to database..." message.
2. It will be replaced by a clean list of exactly 5 notices.
3. Notice that the "Previous" button is greyed out/disabled because you are on Page 1.
4. Click "Next". Watch the data instantly swap to items #6-10, and notice how fast it is (since we already fetched the full 100 items from the server).

State Management with Redux or Zustand

Introduction to Redux concepts (store, actions, reducers)

Redux is the most famous state management tool in React. It completely separates your data from your UI. I like to teach Redux using a **Banking Analogy**:

1. **The Store (The Vault):** One giant, secure JavaScript object that holds all the data for your entire application. You cannot touch it directly.
2. **Actions (The Deposit Slip):** A simple object that describes *what* you want to do (e.g., "Add 100 pesos").
3. **Reducers (The Teller):** A function that takes your Action, goes into the Store, safely updates the data, and returns the new State.

Examples of Core Redux Concepts:

```
// Example A: An Action (The Deposit Slip)
// It must always have a "type" property.
const action = {
  type: 'ADD_CLIENT',
  payload: 'Senator Risa'
};

// Example B: An Action Creator
// A function that builds the action object so you don't have to type it manually every time.
const addClient = (name) => {
```

```
  return { type: 'ADD_CLIENT', payload: name };
};

// Example C: A Reducer (The Teller)
// It checks the action type and calculates the new state.
function clientReducer(state = [], action) {
  if (action.type === 'ADD_CLIENT') {
    return [...state, action.payload]; // Adds the new client to the array
  }
  return state;
}

// Example D: Dispatching (Handing the slip to the teller)
// store.dispatch() is how you trigger the whole process.
// store.dispatch(addClient('Senator Chiz'));
```

Redux Toolkit basics

Classic Redux required *a lot* of boilerplate code. You had to write separate files for actions, reducers, and constants. **Redux Toolkit (RTK)** is the modern, official way to write Redux. It combines actions and reducers into a single concept called a **"Slice."**
(Note: Requires npm install @reduxjs/toolkit react-redux)

Example of Redux Toolkit:

To test this demo, you must first install the libraries in your terminal: **npm install @reduxjs/toolkit react-redux**

Redux requires three pieces: a "Slice" (to define the state and actions), a "Store" (to hold the slices), and a "Provider" (to wrap your app).

1. Create the Slice (State & Reducers):

- **File Placement:** src/store/counterSlice.js

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => {
      // RTK uses Immer under the hood, so we can "mutate" state directly!
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    },
```

```
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    },
  },
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

2. Create the Store:

- **File Placement:** src/store/store.js

```
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});
```

3. Wrap your App and use the Store:

- **File Placement:** src/App.jsx

```
import React from 'react';
// Import Redux tools
import { Provider, useSelector, useDispatch } from 'react-redux';
import { store } from './store/store';
import { increment, decrement, incrementByAmount } from './store/counterSlice';

// A child component that reads and updates the Redux state
function CounterComponent() {
  const count = useSelector((state) => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div style={{ padding: '20px', border: '2px solid purple', maxWidth: '300px' }}>
      <h2>Redux Toolkit Count: {count}</h2>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())} style={{ margin: '0 10px' }}>-</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>Add 5</button>
    </div>
  );
}

// The main App component that Provides the store
function App() {
```

```
  return (
    <Provider store={store}>
      <div style={{ padding: '40px', fontFamily: 'sans-serif' }}>
        <h1>Redux Toolkit Demo</h1>
        <CounterComponent />
      </div>
    </Provider>
  );
}

export default App;
```

Run your app. Click the buttons. Notice how useSelector grabs the data, and useDispatch fires off the actions we defined in our slice.

Zustand as a lightweight alternative

"Zustand" is German for "State." While Redux is powerful, it is often overkill for small-to-medium apps. Zustand has become incredibly popular because it has **zero boilerplate**. There are no "Providers" to wrap your app in, and no massive slice setups. It relies entirely on Custom Hooks.
(Note: Requires npm install zustand)

Example of Zustand:

To test this demo, you must first install the library in your terminal: **npm install zustand**
Zustand is incredibly simple. There are no Providers, no reducers to combine, and no context wrappers.

You just create a custom hook and use it anywhere.

1. Create the Store Hook:

- **File Placement:** src/store/useStore.js

```
import { create } from 'zustand';

// create() takes a callback function that returns the state and actions
export const useStore = create((set) => ({
  // 1. The State
  bears: 0,

  // 2. The Actions (functions that update the state)
  increasePopulation: () => set((state) => ({ bears: state.bears + 1 })),
  removeAllBears: () => set({ bears: 0 }),
}));
```

2. Use the hook directly in your component:

- **File Placement:** src/App.jsx

```
import React from 'react';
import { useStore } from './store/useStore';

function BearCounter() {
  // We can select specific parts of the state to prevent unnecessary re-renders
  const bears = useStore((state) => state.bears);
  return <h2>There are {bears} bears in the forest.</h2>;
}

function Controls() {
  // We can grab the functions directly from our store hook
  const increasePopulation = useStore((state) => state.increasePopulation);
  const removeAllBears = useStore((state) => state.removeAllBears);

  return (
    <div style={{ marginTop: '20px' }}>
      <button onClick={increasePopulation}>Add Bear</button>
      <button onClick={removeAllBears} style={{ marginLeft: '10px' }}>Remove All</button>
    </div>
  );
}

function App() {
  return (
    // Look! No <Provider> wrapper needed for Zustand!
    <div style={{ padding: '40px', fontFamily: 'sans-serif', border: '2px solid orange', maxWidth: '400px',
margin: '20px' }}>
      <h1>Zustand Demo</h1>
      <BearCounter />
      <Controls />
    </div>
  );
}

export default App;
```

Run your app and click the buttons. The Controls component updates the global state, and the BearCounter component instantly reacts to the change. Notice how much less code this took compared to Redux!

Hands-on:

Global state for cart system

Building a Redux Toolkit cart takes about 5 files, whereas Zustand takes exactly 3, making it much easier for students to test in a sandbox.

Step 1: Install Zustand
Stop the server, run npm install zustand, and restart with npm run dev.

Step 2: Create the Global Store (useCartStore.js)
This file lives outside of our components. It acts as the global memory for the cart.

Create a file named **useCartStore.js**.

```
import { create } from 'zustand';

const useCartStore = create((set) => ({
  // 1. The Global State (An array of items)
  items: [],

  // 2. Action: Add an item
  addToCart: (product) => set((state) => ({
    items: [...state.items, product]
  })),

  // 3. Action: Clear the cart
  clearCart: () => set({ items: [] })
}));

export default useCartStore;
```

Step 3: Create the Product List (ProductStore.jsx)
This component allows users to add items to the global cart. Notice we don't need any props!

Create a file named **ProductStore.jsx**.

```
import useCartStore from './useCartStore';

function ProductStore() {
  // We only pull out the addToCart function from our global store
  const addToCart = useCartStore((state) => state.addToCart);

  // Sample inventory using our Senators
  const products = [
    { id: 1, name: "Senator Pia's Running Shoes", price: 2500 },
    { id: 2, name: "Senator Bong's Action Camera", price: 8000 },
```

```
  { id: 3, name: "Senator Imee's Sun Hat", price: 500 }
 ];

 return (
  <div style={{ border: '1px solid gray', padding: '20px', width: '300px' }}>
   <h3>Shop</h3>
   {products.map(product => (
    <div key={product.id} style={{ marginBottom: '15px' }}>
     <strong>{product.name}</strong> - ₱{product.price}
     <br />
     {/* Dispatching the global action */}
     <button onClick={() => addToCart(product)} style={{ marginTop: '5px' }}>
      Add to Cart
     </button>
    </div>
   ))}
  </div>
 );
}
export default ProductStore;
```

Step 4: Create the Cart Display (CartBadge.jsx)
This component is completely disconnected from the ProductStore, yet it will instantly update when items are added because they share the global Zustand store.

Create a file named **CartBadge.jsx**.

```
import useCartStore from './useCartStore';

function CartBadge() {
 // We pull out the items array and the clear function
 const items = useCartStore((state) => state.items);
 const clearCart = useCartStore((state) => state.clearCart);

 // Calculate the total price
 const total = items.reduce((sum, item) => sum + item.price, 0);

 return (
  <div style={{ backgroundColor: '#eee', padding: '20px', width: '300px' }}>
   <h3>Your Cart</h3>
   <p>Items in cart: <strong>{items.length}</strong></p>

   <ul style={{ paddingLeft: '20px' }}>
    {items.map((item, index) => (
     // Using index as key here just for simplicity since duplicates can exist
     <li key={index}>{item.name}</li>
```

```
      ))}
    </ul>

    <h4>Total: ₱{total}</h4>

    {items.length > 0 && (
      <button onClick={clearCart} style={{ color: 'red' }}>Empty Cart</button>
    )}
  </div>
 );
}

export default CartBadge;
```

Step 5: Put it together in App.jsx

Open **App.jsx**, delete everything, and paste this to see them side-by-side:

```
import ProductStore from './ProductStore';
import CartBadge from './CartBadge';

function App() {
 return (
  <div style={{ fontFamily: 'sans-serif', padding: '40px' }}>
    <h2>E-Commerce Dashboard</h2>
    <p style={{ color: 'gray' }}>
      Notice how the Store and the Cart do not pass any props to each other, yet they stay perfectly in sync!
    </p>

    <div style={{ display: 'flex', gap: '40px', marginTop: '20px' }}>
      <ProductStore />
      <CartBadge />
    </div>
  </div>
 );
}

export default App;
```

Performance Optimization

React. memo, useCallback, useMemo

By default, when a Parent component re-renders (like when you type in a search box), *every single Child component* inside it also re-renders, even if their data hasn't changed. This is a massive waste of computer power.
We fix this with **Memoization** (caching or "remembering" things).

- **React.memo:** Wraps a whole Component. It tells React: "Only re-render this child if its props actually changed."
- **useMemo:** Wraps a calculation. It tells React: "This math takes 5 seconds to run. Only run it again if the variables change. Otherwise, just give me the saved answer."
- **useCallback:** Wraps a function. In JavaScript, every time a component re-renders, it creates brand new functions. useCallback saves the function so it doesn't get recreated, which prevents React.memo from accidentally breaking.

Examples of Memoization:
Example 1: React.memo (Skipping renders for static props)
When a parent re-renders, all its children do too—even if their props haven't changed. React.memo wraps a child component so it only re-renders if its specific props change.

StatBox.js

```
import React, { memo } from 'react';

// Wraps the component in memo
const StatBox = memo(({ title, value }) => {
  console.log(` Rendering StatBox: ${title}` );
  return <div className="box"><h2>{title}</h2><p>{value}</p></div>;
});

export default StatBox;
```

App.js

```
import React, { useState } from 'react';
import StatBox from './StatBox';

export default function App() {
  const [timer, setTimer] = useState(0);

  return (
    <div>
      <button onClick={() => setTimer(timer + 1)}>Tick: {timer}</button>
      {/* StatBox won't re-render on every tick, because its props don't change */}
      <StatBox title="High Score" value={9999} />
    </div>
```

```
  );
}
```

Example 2: useCallback (Stabilizing functions passed to children)
If you pass a function to a React.memo child, the child will *still* re-render every time. Why? Because React creates a brand-new function instance on every parent render. useCallback fixes this.

ActionBtn.js

```
import React, { memo } from 'react';

const ActionBtn = memo(({ onClick, label }) => {
  console.log(` Rendering Button: ${label}` );
  return <button onClick={onClick}>{label}</button>;
});

export default ActionBtn;
```

App.js

```
import React, { useState, useCallback } from 'react';
import ActionBtn from './ActionBtn';

export default function App() {
  const [text, setText] = useState("");

  // Memoizes the function so it retains the same memory address across renders
  const handleSubmit = useCallback(() => {
    console.log("Submitted!");
  }, []); // Empty dependency array = never recreate

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      {/* Because handleSubmit is stable, ActionBtn won't re-render when typing */}
      <ActionBtn label="Submit Data" onClick={handleSubmit} />
    </div>
  );
}
```

Example 3: useMemo (Caching expensive calculations)
If you have a heavy math operation or data transformation, you don't want to run it on every single render. useMemo caches the *result* of the function.

App.js

```
import React, { useState, useMemo } from 'react';

export default function App() {
  const [count, setCount] = useState(0);
  const [numbers] = useState([10, 20, 30, 40, 50]); // Imagine this is 10,000 items

  // Only re-runs the loop if the 'numbers' array actually changes
  const total = useMemo(() => {
    console.log("Calculating total...");
    return numbers.reduce((acc, curr) => acc + curr, 0);
  }, [numbers]);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Re-render ({count})</button>
      <p>Total sum is: {total}</p>
    </div>
  );
}
```

Code splitting and lazy loading

When you build a React app, tools like Vite bundle all your JSX into one giant JavaScript file. If a user visits the "Login" page, they still have to download the code for the "Settings" page, the "Dashboard," and the "Admin Panel." This makes the initial website load very slow.

Lazy Loading splits your app into chunks. React only downloads the code for a page when the user actually clicks the link to go there.

Examples of Lazy Loading:
Example 1: Route-based Lazy Loading
The most common and effective way to split code. Only load the JavaScript for a page when the user navigates to it.

Home.js and **Dashboard.js**

```
// Simple placeholder components for the routes
export default function Component() { return <div>Page Content</div>; }
```

App.js

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// 1. Dynamically import the routes
const Home = lazy(() => import('./Home'));
const Dashboard = lazy(() => import('./Dashboard'));

export default function App() {
  return (
    <BrowserRouter>
      <nav><Link to="/">Home</Link> | <Link to="/dashboard">Dash</Link></nav>
      {/* 2. Wrap routes in Suspense to show a fallback while the chunk downloads */}
      <Suspense fallback={<div>Loading Page...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/dashboard" element={<Dashboard />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Example 2: Component-based Lazy Loading (Modals/Drawers)

Don't load the massive code for a complex modal (like an image editor or a rich text editor) until the user actually clicks the button to open it.

HeavyEditor.js

```
export default function HeavyEditor() {
  return <div className="modal">Massive text editor loaded!</div>;
}
```

App.js

```
import React, { useState, Suspense, lazy } from 'react';

// Import only happens when React tries to render it
const HeavyEditor = lazy(() => import('./HeavyEditor'));

export default function App() {
  const [showEditor, setShowEditor] = useState(false);

  return (
    <div>
      <h1>Blog Post</h1>
      <button onClick={() => setShowEditor(true)}>Open Editor</button>
```

```
    {/* The network request for HeavyEditor.js fires ONLY when showEditor is true */}
    {showEditor && (
      <Suspense fallback={<p>Warming up the editor...</p>}>
        <HeavyEditor />
      </Suspense>
    )}
  </div>
 );
}
```

Avoiding unnecessary re-renders

Sometimes you don't even need useMemo. You just need to write smarter React code.

Examples of Avoiding Re-renders:
Example 1: Moving State Down (Colocation)
If a piece of state only affects one specific UI element, don't put it in the parent component. Move it down.

App.js

```
import React, { useState } from 'react';

// BAD: Putting the toggle state in App makes the SlowList re-render every time you open the details.
// GOOD: We moved the toggle state INTO the Details component.
function DetailsToggle() {
 const [isOpen, setIsOpen] = useState(false);
 return (
  <div>
   <button onClick={() => setIsOpen(!isOpen)}>Toggle Details</button>
   {isOpen && <p>Here are some extra details.</p>}
  </div>
 );
}

function SlowList() {
 console.log("Slow list rendered!");
 return <ul><li>Item 1</li><li>Item 2</li></ul>;
}

export default function App() {
 return (
  <div>
   {/* Clicking the toggle no longer forces App (and SlowList) to re-render */}
   <DetailsToggle />
```

```
    <SlowList />
  </div>
 );
}
```

Example 2: Passing Elements as children

If you have an expensive component that *must* sit inside a component with rapidly changing state (like a drag-and-drop container or a scroll tracker), pass it as children. React knows children haven't changed, so it skips rendering them.

App.js

```
import React, { useState } from 'react';

function MouseTracker({ children }) {
 const [pos, setPos] = useState({ x: 0, y: 0 });

 return (
  <div
   onMouseMove={(e) => setPos({ x: e.clientX, y: e.clientY })}
   style={{ height: '200px', background: '#eee' }}
  >
   <p>Mouse: {pos.x}, {pos.y}</p>
   {/* Even though MouseTracker re-renders hundreds of times a second,
      'children' is untouched because it was instantiated in App.js */}
   {children}
  </div>
 );
}

export default function App() {
 return (
  <MouseTracker>
   <div className="expensive-component">I am heavy, but I won't re-render on mouse move!</div>
  </MouseTracker>
 );
}
```

Example 3: Replacing useState with useRef for Background Tracking

If you need to track a value that changes, but you *don't* need the screen to update when it changes, use useRef instead of useState.

App.js

```
import React, { useState, useRef } from 'react';

export default function App() {
```

```
  const [renders, setRenders] = useState(0);

  // Changing this ref DOES NOT trigger a re-render
  const backgroundCounter = useRef(0);

  const handleRefClick = () => {
    backgroundCounter.current += 1;
    console.log("Ref is now:", backgroundCounter.current);
  };

  return (
    <div>
      <button onClick={() => setRenders(renders + 1)}>
        Force Re-render (Renders: {renders})
      </button>

      {/* Clicking this button mutates data perfectly, but doesn't waste React's render cycles */}
      <button onClick={handleRefClick}>
        Increment Ref Value (See Console)
      </button>
    </div>
  );
}
```

Hands-on:

Optimize a slow React app

We are going to build an app with a severe performance problem, and then fix it.

Imagine a banking app that has a massive list of transactions. We also have a simple text input for a "Status Update." If we type in the input, the *entire* heavy transaction list re-renders on every keystroke, making typing feel laggy and terrible.

Step 1: Create the Heavy Component (TransactionHistory.jsx)

1. Create a new file named **TransactionHistory.jsx**.
2. Paste this code. It includes an artificial "slowdown" loop to simulate thousands of calculations.

```
import React from 'react';

// 1. Notice we are NOT using React.memo yet!
function TransactionHistory({ clientName }) {

  // ARTIFICIAL SLOWDOWN: Simulating a heavy calculation that takes time
  let startTime = performance.now();
  while (performance.now() - startTime < 30) {
```

```
  // Do nothing for 30 milliseconds to lag the computer
 }

 console.log(` Heavy Component Re-rendered for ${clientName}` );

 return (
  <div style={{ border: '2px solid red', padding: '10px', marginTop: '20px' }}>
   <h4>Ledger for: {clientName}</h4>
   <p>Loaded 10,000 complex transaction records.</p>
  </div>
 );
}
// Export it normally
export default TransactionHistory;
```

Step 2: The "Slow" Dashboard (App.jsx)

1. Open your **App.jsx**.
2. Delete everything and paste this:

```
import { useState } from 'react';
import TransactionHistory from './TransactionHistory';

function App() {
 const [notes, setNotes] = useState("");
 const [client, setClient] = useState("Senator Pia");

 return (
  <div style={{ padding: '30px', fontFamily: 'sans-serif', maxWidth: '500px' }}>
   <h2>Banking Dashboard</h2>

   {/* 1. The input box. Every time we type a letter, the state updates. */}
   {/* When the state updates, the ENTIRE App component re-renders! */}
   <label>Type a quick note (Watch the console):</label>
   <input
    type="text"
    value={notes}
    onChange={(e) => setNotes(e.target.value)}
    style={{ width: '100%', padding: '8px', marginBottom: '10px' }}
   />
   <p>Draft: {notes}</p>

   {/* 2. Because App re-renders, this Heavy component is forced to re-render,
       causing our typing to lag! */}
   <TransactionHistory clientName={client} />
  </div>
 );
```

```
}

export default App;
```

Step 3: Test the Problem
Save both files. Open your browser AND open the browser's Developer Tools Console (F12 or Ctrl+Shift+I).

- Start typing very fast in the "Quick note" box.
- You will notice the typing feels slightly sluggish.
- Look at the Console. You will see Heavy Component Re-rendered printed 20 times (once for every single letter you typed!). We are wasting computer power on a component that hasn't changed.

Step 4: The Optimization (The Fix)
Let's fix it using React.memo.

1. Go back to **TransactionHistory.jsx**.
2. Change the very last line of the file from: export default TransactionHistory; To this: export default React.memo(TransactionHistory);
3. Save the file.

Step 5: Test the Solution
Go back to your browser, clear the console, and type in the input box again.

- The typing is now lightning-fast!
- Look at the console. The heavy component renders *exactly once* when the page loads, and **never again** while you type. React.memo successfully protected it because the clientName prop ("Senator Pia") never changed.

Testing in React

Jest basics

Jest (or Vitest) is the "engine." It is a JavaScript testing framework that runs your tests, checks your logic, and prints out a Pass/Fail report. It doesn't know anything about React or HTML; it only knows standard JavaScript.

If you are starting completely from scratch (not using a starter template), you will need to install Jest and React Testing Library first. You can do that by running:

```
npm install --save-dev jest @testing-library/react @testing-library/jest-dom @testing-library/user-event jest-environment-jsdom
```

Then, add this to the "scripts" section of your package.json:

```
"scripts": {
  "test": "jest"
}
```

We use **Assertions** to test our code. We "expect" a value "to be" something.

Examples of Jest Assertions:
Example 1: Basic Assertions (Testing pure functions)
You provide inputs and expect a specific output.

math.js

```
export const add = (a, b) => a + b;
export const isEven = (num) => num % 2 === 0;
```

math.test.js

```
import { add, isEven } from './math';

describe('Math Utilities', () => {
  it('should add two numbers correctly', () => {
    // Asserting exact equality
    expect(add(2, 3)).toBe(5);
  });

  it('should correctly identify even numbers', () => {
    // Asserting boolean values
    expect(isEven(4)).toBeTruthy();
    expect(isEven(5)).toBeFalsy();
  });
});
```

Example 2: Testing Asynchronous Code
When your code fetches data or uses timeouts, your tests need to wait for those operations to finish before making assertions.

fetcher.js

```
export const fetchGreeting = () => {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Hello, World!"), 100);
  });
};
```

fetcher.test.js

```
import { fetchGreeting } from './fetcher';

describe('Async Fetcher', () => {
  it('should resolve with a greeting using async/await', async () => {
    const data = await fetchGreeting();
    expect(data).toBe("Hello, World!");
  });
```

```
  // Alternative Jest syntax for promises
  it('should resolve using resolves matcher', () => {
    return expect(fetchGreeting()).resolves.toBe("Hello, World!");
  });
});
```

Example 3: Mocking Functions (jest.fn)
Sometimes you want to test if a callback was triggered, rather than testing the callback's logic itself. jest.fn() creates a "spy" that remembers how it was used.

processor.js

```
export const processItems = (items, callback) => {
  items.forEach(item => callback(item.toUpperCase()));
};
```

processor.test.js

```
import { processItems } from './processor';

describe('Processor', () => {
  it('should call the callback for each item', () => {
    // Create a mock function
    const mockCallback = jest.fn();

    processItems(['apple', 'banana'], mockCallback);

    // Assert the mock was called the correct number of times
    expect(mockCallback).toHaveBeenCalledTimes(2);

    // Assert what arguments the mock received
    expect(mockCallback).toHaveBeenCalledWith('APPLE');
    expect(mockCallback).toHaveBeenCalledWith('BANANA');
  });
});
```

React Testing Library

If Jest is the engine, React Testing Library (RTL) is the steering wheel. RTL renders your React components in an invisible, "fake" browser so Jest can look at them.

You should test your app exactly how a user interacts with it. A user doesn't know what a useState variable is. They only know what they can see (Text) and what they can click (Buttons). Therefore, RTL forces you to find elements by their visible text or roles.

Examples of React Testing Library:

Example 1: Rendering and Querying the DOM

Verify that a component renders the expected content based on the props it receives.

WelcomeMessage.js

```
import React from 'react';

export default function WelcomeMessage({ name }) {
  return <h1>Welcome to the app, {name}!</h1>;
}
```

WelcomeMessage.test.js

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import '@testing-library/jest-dom'; // Provides custom matchers like toBeInTheDocument
import WelcomeMessage from './WelcomeMessage';

it('renders a personalized welcome message', () => {
  render(<WelcomeMessage name="Alice" />);

  // Find the element by the text the user would see
  const heading = screen.getByText('Welcome to the app, Alice!');

  expect(heading).toBeInTheDocument();
});
```

Example 2: User Interaction (fireEvent)

Simulate user actions like clicking, typing, or submitting forms to ensure the UI updates correctly.

ToggleButton.js

```
import React, { useState } from 'react';

export default function ToggleButton() {
  const [isOn, setIsOn] = useState(false);
  return (
    <button onClick={() => setIsOn(!isOn)}>
      {isOn ? 'Turn Off' : 'Turn On'}
    </button>
  );
}
```

ToggleButton.test.js

```
import React from 'react';
import { render, screen, fireEvent } from '@testing-library/react';
import '@testing-library/jest-dom';
```

```
import ToggleButton from './ToggleButton';

it('toggles text when clicked', () => {
  render(<ToggleButton />);

  const button = screen.getByRole('button', { name: 'Turn On' });
  expect(button).toBeInTheDocument();

  // Simulate a user click
  fireEvent.click(button);

  // Assert that the UI updated
  expect(screen.getByRole('button', { name: 'Turn Off' })).toBeInTheDocument();
});
```

Example 3: Testing Asynchronous UI Updates
When a component fetches data on mount, you must wait for the UI to update before asserting.

UserProfile.js

```
import React, { useState, useEffect } from 'react';

export default function UserProfile() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // Simulating an API call
    setTimeout(() => setUser({ name: 'John Doe' }), 100);
  }, []);

  if (!user) return <p>Loading...</p>;
  return <div><h2>{user.name}</h2></div>;
}
```

UserProfile.test.js

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import '@testing-library/jest-dom';
import UserProfile from './UserProfile';

it('shows loading then displays the user', async () => {
  render(<UserProfile />);

  // Synchronous check for initial state
  expect(screen.getByText('Loading...')).toBeInTheDocument();
```

```
  // findBy queries are async and automatically wait (up to 1000ms by default)
  const userName = await screen.findByText('John Doe');
  expect(userName).toBeInTheDocument();
});
```

Snapshot testing

Snapshot testing is like taking a photograph of your component's HTML output.

When you run a snapshot test for the first time, Jest creates a text file containing the exact HTML structure of your component. Every time you run tests after that, it takes a *new* picture and compares it to the saved one. If a developer accidentally deletes a CSS class or an HTML tag, the test fails, warning you that the UI changed unexpectedly.

Examples of Snapshot Testing:
Example 1: Standard Component Snapshots
Ensure your UI structure doesn't change unexpectedly. If you alter the HTML, the test will fail, prompting you to either fix the regression or update the snapshot.

Badge.js

```
import React from 'react';

export default function Badge({ text, color = 'blue' }) {
  return <span className={` badge bg-${color}` }>{text}</span>;
}
```

Badge.test.js

```
import React from 'react';
import { render } from '@testing-library/react';
import Badge from './Badge';

it('matches the snapshot for a default badge', () => {
  // Extract the raw DOM node using the 'container' property
  const { container } = render(<Badge text="New" />);

  // This creates a __snapshots__ folder with the saved HTML output
  expect(container.firstChild).toMatchSnapshot();
});
```

Example 3: Inline Snapshots
Instead of saving the snapshot to an external file, Jest writes the snapshot *directly into your test file* as an argument. This is amazing for readability.

format.js

```
export const formatCurrency = (amount) => `$${amount.toFixed(2)}`;
```

format.test.js

```
import { formatCurrency } from './format';

it('formats numbers as currency', () => {
  // When you first run this with expect().toMatchInlineSnapshot(),
  // Jest will automatically rewrite this file to insert the expected string.
  expect(formatCurrency(10.5)).toMatchInlineSnapshot(`"$10.50"`);
  expect(formatCurrency(42)).toMatchInlineSnapshot(`"$42.00"`);
});
```

To run the test(s):

```
# The Standard Run
npm test

# Watch Mode (Recommended for Active Development)
# Watch mode keeps Jest running in the background. Every time you save a file, Jest detects the changes
#and only re-runs the tests related to the files you just edited. This is the fastest way to work.
# If your standard npm test doesn't automatically open in watch mode, you can force it:
npm test -- --watchAll

# Updating Snapshots
# If you intentionally change a component's UI (like changing a button's color from blue to red), your
# snapshot test will fail because the new HTML doesn't match the old saved picture.
# To tell Jest, "This change was intentional, please update the saved snapshot," you run the update
# command:
npm test -- -u
(You can also usually just press u on your keyboard if you are currently running Jest in Watch Mode).

# Checking Code Coverage
# Jest has a fantastic built-in tool that checks how much of your actual code is covered by your tests. It
# will generate a report showing exactly which lines of code your tests missed.
# This will print a table in your terminal and also create a coverage folder in your project. You can open
# coverage/lcov-report/index.html in your browser for a beautiful, clickable UI of your code coverage.
npm test -- --coverage
```

Hands-on:

Write unit tests for components

We are going to build a VoteCounter component, and then write a test file to prove that the initial text is correct and that the math works when the button is clicked.

Step 1: The Component (VoteCounter.jsx)
Create a new file in your src folder named **VoteCounter.jsx** and paste this code:

```
import { useState } from 'react';

function VoteCounter({ candidateName }) {
  const [votes, setVotes] = useState(0);

  return (
    <div style={{ padding: '20px', border: '1px solid black', width: '300px' }}>
      {/* We use specific tags (h2, button) so RTL can easily find their "Roles" */}
      <h2>Candidate: {candidateName}</h2>

      <p>Current Votes: <span data-testid="vote-count">{votes}</span></p>

      <button
        onClick={() => setVotes(votes + 1)}
        style={{ backgroundColor: 'blue', color: 'white', padding: '10px' }}
      >
        Cast Vote
      </button>
    </div>
  );
}

export default VoteCounter;
```

Step 2: The Test File (VoteCounter.test.jsx)
In React, test files usually live right next to the component they are testing and end in .test.jsx.

Create a file named **VoteCounter.test.jsx** and paste this code:

```
import { render, screen, fireEvent } from '@testing-library/react';
import VoteCounter from './VoteCounter';

// 'describe' creates a testing suite (a folder to group related tests together)
describe('VoteCounter Component', () => {

  // Test 1: Does it render the props correctly?
  test('displays the correct candidate name', () => {
```

```
  // 1. Render it in the fake browser
  render(<VoteCounter candidateName="Senator Lito" />);

  // 2. Look for the heading
  const heading = screen.getByRole('heading', { name: /Candidate: Senator Lito/i });

  // 3. Assert it is there
  expect(heading).toBeInTheDocument();
 });

 // Test 2: Does the initial state start at zero?
 test('starts with 0 votes', () => {
  render(<VoteCounter candidateName="Senator Lito" />);

  // We can use a custom data-testid if text is hard to target
  const countDisplay = screen.getByTestId('vote-count');

  expect(countDisplay.textContent).toBe("0");
 });

 // Test 3: Does the button actually change the state? (Simulating user behavior)
 test('increments the vote count when the button is clicked', () => {
  render(<VoteCounter candidateName="Senator Lito" />);

  // Find the button and the display
  const button = screen.getByRole('button', { name: /Cast Vote/i });
  const countDisplay = screen.getByTestId('vote-count');

  // Simulate clicking the button ONE time
  fireEvent.click(button);

  // Assert that the text changed from 0 to 1
  expect(countDisplay.textContent).toBe("1");

  // Click it again!
  fireEvent.click(button);
  expect(countDisplay.textContent).toBe("2");
 });

});
```

How to Run This in Class:

If you are using Vite, test execution isn't baked into the standard npm run dev command.

1. In the terminal, you would ensure testing tools are installed: npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom

2. You run the test by typing: npx vitest
3. The terminal will output a green checklist showing that all 3 tests passed

Deploying React Apps

Building for production

Up until now, your students have been using npm run dev. This runs a "Development Server" which is purposely slow because it includes debugging tools, error overlays, and hot-reloading.

Before putting an app on the internet, you must "Build" it. The build process takes all your JSX, React libraries, and CSS, and compresses them into tiny, highly optimized standard HTML and JavaScript files that browsers can load instantly.

Key Concepts of the Build Process:

```
# Example A: The Build Command
# Running this in the terminal tells Vite to start the optimization engine.
npm run build

# Example B: The Output Folder
# After running the command, a new folder appears called "dist" (Distribution).
# THIS is the only folder you actually upload to the internet!

# Example C: Testing the Build Locally
# You can test the compressed version before uploading it.
npm run preview

# Example D: What happens to the code?
# The build process "minifies" the code, turning this:
function Greeting() { return <h1>Hello Senator Risa</h1>; }
# Into something like this (to save file size):
function e(){return r("h1",null,"Hello Senator Risa")}
```

Environment variables

When your app connects to a live database or a third-party API, you often need secret keys or passwords. **You must never hard-code secrets directly into your React files**, or anyone can right-click "View Source" on your website and steal them.

Environment variables act as a hidden safe. You store the secrets in a special file on your computer, and the hosting provider injects them securely when the app goes live.

Examples of Environment Variable Usage:

```
// Example A: The .env File
// Create a file named exactly ".env" in the root of your project.
// In Vite, all React variables MUST start with "VITE_".
VITE_API_KEY=12345XYZ
VITE_DATABASE_URL=https://api.bangko-sentral.ph

// Example B: The .gitignore Rule
// Your .gitignore file must contain ".env" so you don't accidentally upload it to GitHub!

// Example C: Reading the Variable in React
// Instead of hardcoding the key, you call it dynamically.
function Dashboard() {
  const apiKey = import.meta.env.VITE_API_KEY;
  return <p>Connected with key: {apiKey}</p>;
}

// Example D: Using it in an API Fetch
const fetchClients = async () => {
  const url = import.meta.env.VITE_DATABASE_URL;
  const res = await fetch(` ${url}/clients/Chiz` );
};
```

Deploying to Netlify, Vercel, or GitHub Pages

Traditional web hosting (like GoDaddy or HostGator) can be complicated for modern React apps. Today, the industry relies on specialized "Cloud Edge" hosts.

- **Vercel:** The absolute gold standard for React. It is lightning-fast, automatically optimizes assets, and has incredible GitHub integration.
- **Netlify:** Extremely similar to Vercel. It is famous for its "Drag and Drop" feature, making it the easiest platform for beginners who haven't learned Git yet.
- **GitHub Pages:** Free and built into GitHub, but it requires confusing configuration for React Router to work properly. Not recommended for complex Single Page Applications.

Hands-on:

Deploy a working project to a hosting service (Netlify)

We will teach your students the absolute fastest way to get their banking app on the internet using **Netlify Drop**. This requires zero accounts, zero Git knowledge, and takes about 60 seconds.

Step 1: Prepare the App
Before uploading, we need to generate the optimized dist folder.

1. Open your project in the **IDE**.
2. Open the integrated terminal at the bottom.

3. Stop your local server (Ctrl + C).
4. Type npm run build and hit Enter.
5. Watch the terminal. It will print out a list of files it compressed. Look at your file explorer on the left: a new folder named **dist** has appeared!

Step 2: The Netlify Drop

1. Open your web browser and go to **app.netlify.com/drop**.
2. Open your computer's regular file explorer (Windows Explorer or Mac Finder).
3. Find your my-react-app folder.
4. Drag *only* the **dist** folder and drop it into the circle on the Netlify webpage.

Step 3: Claim Your URL

1. Netlify will take about 5 seconds to upload the files.
2. It will generate a live, public URL (something like https://cheerful-tarsier-8a9b2c.netlify.app).
3. Click the link! Your React app is now live on the internet, accessible from any phone or computer in the world.

(Tip: To change the random name to something professional like senate-banking-app.netlify.app, you can create a free Netlify account and click "Site Settings" -> "Change site name".)

Deploy a working project to a hosting service (Vercel)

Method 1: The Professional Way (GitHub Integration)

This is how real-world teams operate. You connect Vercel to your GitHub account. Every time you push an update to your code on GitHub, Vercel detects it and automatically rebuilds and updates your live website in seconds.

Step 1: Push Code to GitHub

Ensure your students have initialized a Git repository and pushed their React project to a repository on their GitHub account. *(Note: They do not need to push the node_modules or dist folders; Git ignores these by default)*.

Step 2: Import to Vercel

1. Go to **vercel.com** and create a free account using "Continue with GitHub".
2. Once logged in, click the black **"Add New..."** button and select **"Project"**.
3. Vercel will show a list of your GitHub repositories. Find the banking app repository and click **"Import"**.

Step 3: Configure and Deploy

1. Vercel is incredibly smart. It will automatically detect that you are using Vite and React.
2. It will pre-fill the "Build Command" (npm run build) and the "Output Directory" (dist).
3. Expand the **"Environment Variables"** section if you have any secret API keys (like the ones we discussed in the previous module) and paste them here.
4. Click the big **"Deploy"** button.
5. Wait about 30 seconds for the confetti animation, and click the generated live URL!

Method 2: The Terminal Way (Vercel CLI)
If your students haven't learned Git yet, or if you just want to deploy instantly without leaving your code editor, the Vercel Command Line Interface (CLI) is pure magic.

Step 1: Install the CLI

1. Open your integrated terminal.
2. Run this command to install the Vercel tool globally on your computer: npm install -g vercel

Step 2: Run the Deploy Command

1. Ensure your terminal is inside your my-react-app project folder.
2. Type this single word and hit Enter: vercel

Step 3: Follow the Prompts

The terminal will ask you a series of quick questions. Here is how your students should answer them:

- **Set up and deploy?** Y (Yes)
- **Which scope do you want to deploy to?** (Hit Enter to select your default account)
- **Link to existing project?** N (No)
- **What's your project's name?** (Hit Enter to accept the default, e.g., my-react-app)
- **In which directory is your code located?** (Hit Enter to accept ./)
- **Want to modify these settings?** N (No, Vercel automatically detects Vite!)

Once the upload finishes, the terminal will print out a "Production URL". Click it, and the app is live.